

Integrating Topology and Traffic Engineering to Maximize Throughput in Reconfigurable Networks

Chen Griner^a, Chen Avin^a

^a*School of Electrical and Computer Engineering, Ben Gurion University of the Negev, Israel*

Abstract

The state-of-the-art topologies of datacenter networks are fixed, based on electrical switching technology, and by now, we understand their throughput and cost well. For the past years, researchers have been developing novel optical circuit switching technologies that enable the emergence of reconfigurable datacenter networks (RDCNs) that support dynamic physical topologies. The art of network design of dynamic topologies, i.e., ‘Topology Engineering,’ is still in its infancy. Different designs offer distinct advantages, such as faster switch reconfiguration times or the ability to reconfigure the topology according to the demand, and to date, it is unclear which design maximizes the throughput. This paper aims to improve our analytical understanding and formally studies the throughput of reconfigurable networks by presenting a general and unifying model for dynamic networks and their topology and traffic engineering. We use our model to study demand-oblivious and demand-aware systems and prove new lower bounds for the throughput of a system as a function of its topology and traffic schedules. Next, we offer a novel algorithm for a recent system design that combines both demand-oblivious and demand-aware schedules, and, for the first time, prove this design’s throughput supremacy under a large family of demand matrices with our algorithm. We evaluate this design numerically for sparse and dense traffic and show that our approach can outperform other designs by up to 25% using common network parameters.

Note: This is the author’s accepted manuscript of an article published in *Computer Networks* (Elsevier, 2025).

The final version is available at: doi.org/10.1016/j.comnet.2025.111803

© 2025. This manuscript version is made available under the CC-BY-NC-ND 4.0 license:
creativecommons.org/licenses/by-nc-nd/4.0

Keywords: Datacenter Networks, Optical networks, Network architecture, Topology Engineering

1. Introduction

Datacenter networks (DCNs) are an essential infrastructure for modern internet applications, which are at the base of our digital lives. Large-scale web services, cloud computing, and the recent proliferation of large language models (LLMs) are just a few of the typical uses of datacenters. The use of different online platforms and services is expected to increase even further in the coming years, resulting in an explosive increase in demand for datacenters [1, 2]. Therefore, DCNs must keep improving their performance by

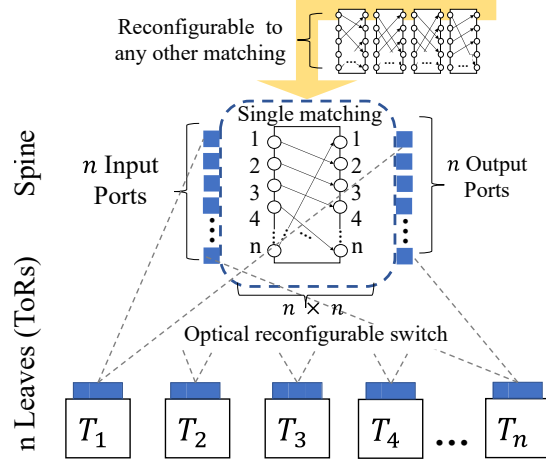


Figure 1: A schematic view of our simplified network model. A two-layer leaf-spine architecture. The spine layer consists of a single switch that can adapt and change its matching. The dashed lines represent unidirectional links.

adopting new ideas and technologies. Traditionally, DCNs have been connected through electrical packet switches in a wired static topology, e.g., fat-trees, clos networks [3, 4, 5], and expanders [6]. In these DCNs, since the topology is static, it needs to function adequately under any traffic pattern, and, therefore, the topology is optimized for a worst-case traffic pattern and is oblivious to the actual traffic. Static DCNs, therefore, often employ what is termed “*Traffic engineering*” and reroute traffic to avoid congestion and increase overall efficiency and throughput [7, 8].

However, electrical packet-switched networks are just one technology used for datacenter networks. Recent years have given rise to optical circuit switches (OCSes), a newer technology with new possibilities [9, 10, 11]. OCS and other switching technologies, such as those based on wavelength gratings as in Sirius [12], enable dynamic reconfigurable network topologies via optical circuit switching. Using these optical switches, the topology can change dynamically in relatively short periods and adapt to different traffic patterns, thus facilitating the emergence of reconfigurable datacenter networks (RDCNs) [9, 13]. RDCNs require what was recently termed “*Topology engineering*” [10]: a new dimension of network design that controls the dynamic topology of the network. Essentially, reconfigurable topologies allow flows to route via shorter paths, increasing throughput by saving network capacity at the cost of a temporal penalty. In other words, these systems can save *bandwidth tax* resulting from multi-hop routing at the cost of a *latency tax* resulting from topology reconfiguration time [14, 15, 16]. Numerous reconfigurable network designs have been proposed [12, 17, 18, 19, 17, 20, 21]. However, a comprehensive understanding of their trade-offs, performance, and limitations under different conditions is still needed. In this work, we propose a unified framework that integrates both traffic and topology engineering to analyze the throughput of these designs. We study three cases of dynamic topologies, which represent state-of-the-art systems. Through rigorous analysis, combining theoretical and empirical approaches, we identify the superior system.

We follow the approach of previous works [22, 14, 16, 17], and model an RDCN as a two-layer leaf-spine topology. Our network model is illustrated schematically in Figure 1. The n nodes of the leaf layer are connected using a *single* optical spine switch with n input and n output ports of equal capacity. At any given time, the switch can hold, and later reconfigure itself into, any directed *matching* between its input and output ports, creating a directed topology (i.e., graph) between the set of nodes. Since, at any given time, the switch implements a single matching, any node could be connected to any other node, but only to one. We further assume that only a single-hop routing is allowed on each matching. However, in our model, data may be sent to a node in one matching and then forwarded to its destination in a different matching. This enables the topology to simulate networks of an arbitrary radix (degree) over time. This switch can reconfigure itself to hold a different matching at any time. The reconfiguration process of a switch, from one matching to another, has a temporal penalty, termed the *reconfiguration time*. Usually, a parameter that depends on the system type. The sequence of matching that a system implements is denoted as *topology schedule*, and it explicitly defines the dynamic topology of the system. Since only single-hop routing is allowed on each matching, we use a traffic engineering approach and determine which traffic (e.g., packets) will be sent over which link and when. This is determined via a *traffic schedule*, which explicitly defines the traffic sent on each matching. In our model, multi-hop routing is the main contributor to bandwidth tax, and reconfiguration time serves as the primary contributor to latency tax. Note that while multi-hop paths increase latency in a realistic setting, we negate this in our abstract model, as we do not consider queues, propagation delay, etc. We assume these effects are relatively minor to the increase caused by the reconfiguration times. The networked system’s throughput can be increased by reducing the “payment” for both tax types. We discuss other details of our network model in Section 2.

There are two basic approaches to topology scheduling (and, more generally, topology engineering) of the RDCNs that our model represents: *demand-aware* and *demand-oblivious* topology designs [13]. The two design approaches use different strategies to achieve lower “taxation”. In a nutshell, demand-aware topologies or what may also be called *adaptive topologies* are dynamic networks that adjust the topology to the actual demand, optimizing the topology to particular traffic patterns by rewiring connections. On the contrary, demand-oblivious topologies [11] present the same topology to any traffic patterns, oblivious to the actual demand. Note, however, that demand-oblivious systems can be dynamic (e.g., RotorNet [14]) or static (e.g., Clos [3]). In this paper, we study both demand-oblivious and demand-aware dynamic topologies.

There are many demand-aware topologies [17, 20, 21, 19, 23]. In these works, the topology can form an arbitrary schedule of matchings where all or most traffic flows are served in a direct connection, i.e., single hop, thus paying no bandwidth tax. Some of the best-known approaches for demand-aware topology scheduling (e.g., Eclipse [18], and Solstice [20]) are based on Birkhoff von Neumann (BvN) decomposition [24, 25] of the demand matrix. Where the demand matrix represents the demand between each pair of nodes

(typically ToRs) in the network.

The BvN decomposition is a well-known and widely studied [18, 26, 27] method used to decompose a (doubly stochastic) demand matrix into a set of permutation matrices, i.e., matchings (see preliminaries in Appendix A). The network can then use output matchings of the BVN decomposition as a topology schedule, serving the demand. We denote systems from this family as Birkhoff von Neumann systems or BvN-SYS. An OCS switch can implement the topology schedule generated using BvN by reconfiguring itself to each permutation matrix in the decomposition in some order. Each reconfiguration incurs a delay in the OCS, which prevents it from transmitting data for a limited time. Therefore, this period, denoted as the *reconfiguration time*, is a main limiting factor for the throughput of these systems. Bounds on the reconfiguration time are varied. Most BvN-SYS type systems need to reconfigure physical connections (e.g., move mirrors), which results in a delay on the scale of *nano-seconds* [12]. More critically, there is a need to calculate a computationally expensive decomposition to serve as a topology schedule. Indeed, improving the run time of the BvN decomposition is the subject of some recent works [18, 26, 27].

In contrast to the demand-aware topologies we just described, demand-oblivious system designs can achieve much faster reconfiguration times (e.g., Sirius [12], RotorNet [14], and Opera [22]). These networks achieve *nano-second* scale reconfiguration times and pay very little in the form of latency tax [12, 16]. The topology schedule approach in these designs is to use a set of *predefined matchings* and quickly ‘rotate’ between them in a *round-robin* and periodic manner, which does not change. This leads to faster reconfiguration since there is no need to calculate a new topology schedule for each demand pattern. In these systems, the union of the schedule’s matchings results in a complete graph¹. All nodes will be connected directly once in each *cycle* (or ‘round’). Due to the round-robin nature of the operation of systems such as RotorNet and Opera, in this paper, we will denote systems from this family as round-robin systems or, RR-SYS. Since reconfiguration time is lower in round-robin systems and the topology configuration is demand-oblivious, one avenue left to improve the throughput of such systems is via *traffic engineering*. In particular, demand-oblivious topology schedules typically use two-hop paths for load balancing, trading a higher bandwidth tax with a lower latency tax to help improve the system’s throughput. For example, Valiant load balancing [29] as used in Sirius [12], or RotorLB, used in RotorNet [14]. We further note that a traffic schedule could be either oblivious (as oblivious routing [30]) or adaptive (as adaptive routing [31]). For example, Sirius [12] uses oblivious routing over oblivious topology while RotorNet [14] and Opera [22] uses adaptive routing over oblivious topology.

Recent proposals have begun to explore what we refer to as *composite* systems (i.e., hybrid topology systems). These systems feature topology schedules that integrate *both* round-robin (demand-oblivious) and

¹Recent work proposed round-robin systems that emulate other regular graphs like expanders [28]. Here, we assume they emulate a complete graph.

System	Traffic engineering	Topology engineering	System DCT ($M(v, u)$)	System DCT (General)	Modeled after
BvN-SYS	Demand-aware	Demand-oblivious	Equation 14	Observation 1	Eclipse & others [18, 17, 20]
RR-SYS	Demand-oblivious	Demand-aware	Equation 15	Theorem 5	RotorNet & others [14, 12, 22]
COMP-SYS	Both	Both	Theorem 8	Unknown	Cerberus & others [16, 19, 32]

Table 1: A summary of each system. The system DCT of COMP-SYS is unknown for a general double stochastic matrix.

BvN (demand-aware) designs into a single schedule. We denote these systems as COMP-SYS. Cerberus [\[16\]](#) showed that a hybrid approach could help to improve the throughput in some important cases, a conclusion shared in other related works on COMP-SYS type systems [\[32, 19\]](#).

In this paper, our goal is to study the demand completion time (DCT) of different systems (i.e., RR-SYS, BvN-SYS, COMP-SYS). In this context, the DCT of a system (or simply, *system DCT*) is defined as the completion time of the *worst-case* demand matrix for that system. This measure guarantees an upper bound on the completion time for *any* demand matrix. This approach allows us to study throughput indirectly, as DCT is inversely proportional to throughput [\[33, 16, 15\]](#), an upper bound on the DCT of a system provides a lower bound on the system’s throughput for *any* demand matrix. In particular, a system with a better DCT than another system will also have better throughput.

In [Table 1](#), we summarize the major elements of each system type at a glance: their traffic or topology engineering and the systems on which each system is modeled. We also reference each system’s *system DCT* on general double stochastic matrices and a subset of double stochastic matrices, $M(v, u)$, which we later describe. We note that the general case is unknown for COMP-SYS type systems.

1.1. Our Contributions

Our main conjecture is that composite systems achieve higher throughput than RR-SYS and BvN-SYS type systems. To validate this, we take a broader perspective and try to understand the strengths and weaknesses of different systems. We show that an COMP-SYS type system performs best for demand patterns that represent worst-case scenarios for both RR-SYS and BvN-SYS systems. Building on this insight, we propose a greedy algorithm that exploits the advantages of each system type and ensures that COMP-SYS type systems have superior performance. We believe the approach outlined by our algorithm will be the core strategy used to make COMP-SYS systems effective in the future. More specifically, our main contributions are the following.

Novel Algorithms and Analysis of COMP-SYS DCT. We propose a novel algorithm for COMP-SYS on an input of general double stochastic matrices and, for the first time, give a detailed result for the system DCT of a composite system on a specific subset of double stochastic matrices in [Theorem 8](#). Our result shows that, under mild assumptions, the system DCT of COMP-SYS is better (lower) than either RR-SYS or BvN-SYS.

Analysis of RR-SYS and BvN-SYS DCTs. We perform a comprehensive analysis of the system DCT

of RR-SYS and BVN-SYS. In particular, we provide the system DCT for BVN-SYS in [Observation 1](#), and for RR-SYS in [Theorem 5](#) for general double stochastic matrices. To do so, we expand previous results and offer new insights and novel bounds on the performance of round-robin systems.

Problem Formalism. We provide a formal theoretical framework integrating topology and traffic engineering to analyze different dynamic topologies. Our framework allows us to define the *system DCT* ([Definition 3](#)) as a function of the topology and traffic scheduling. Our approach also allows us to define the system throughput as the reciprocal of the system DCT ([Equation 2](#)).

Empirical Evaluation. We empirically evaluate our algorithm’s performance on known traffic models using a set of plausible parameters. We show that the COMP-SYS system is superior, by more than 20% in terms of throughput, to any of the other systems, RR-SYS and BVN-SYS, that represent state-of-the-art systems.

Finally, as a contribution to the research community and to facilitate future research work, all our source code was made available online together with this paper [\[34\]](#).

The rest of this paper is organized as follows: We begin by setting up our model in [Section 2](#). This section explains our demand and network model, gives a formal definition of traffic and topology schedules, and provides our problem definition. In the following three sections, we define and derive DCT bounds for our three systems: [Section 3](#) covers BVN-SYS, [Section 4](#) covers RR-SYS and then [Section 5](#) covers the composite system, COMP-SYS. In [Section 6](#) we give a case study that analyzes a subset of double stochastic matrices. This subset captures the worst case demand pattern of both BVN-SYS and RR-SYS, and allows us to demonstrate how COMP-SYS outperforms both. In [Section 7](#), we present an empirical evaluation of our systems for general double stochastic matrices based on a known traffic model. In [Section 8](#) we discuss the practicality of our model. We discuss related work in [Section 9](#). We conclude with a discussion in [Section 10](#). We note that this work builds upon a rigorous theoretical framework. To maintain a concise discussion of our main results and observations, some of the technical details, definitions and proofs were moved to the Appendix, starting from [Appendix A](#).

2. Demand Completion Times (DCT): model and problem definition

In this section, we discuss and define the problem of *demand completion time* (DCT) minimization in reconfigurable dynamic networks. We start by defining our problem’s input, a demand matrix M , representing our network traffic. In turn, we define two types of schedulers: a *topology scheduler* and a *traffic scheduler* (where we consider the special case that allows at most two hop paths). Together, both schedulers can formally describe the forwarding of traffic in a dynamic network and the length of the topology scheduler (in seconds), which in turn defines the matrix DCT. Finally, In [Section 2.3](#), we define the DCT minimization problem as finding the best schedulers for a given system, where each such scheduler’s DCT is defined by its worst-case demand matrix.

2.1. Demand Matrix and the Network Model

Our network is modeled in a similar manner to the ToR-Matching-ToR (TMT) model, which was recently the basis for several RDCNs architectures [14, 12, 16, 19]. The network illustrated schematically in Figure 1, comprises two layers. A spine switch layer and a leaf switch layer. The leaf layer has n nodes representing top-of-the-rack (ToR) switches $T_1, \dots, T_n$, and the spine layer has a *single* reconfigurable (dynamic) switch. The single switch is a special case of the TMT. The single switch abstraction was also used in previous works [9, 35, 12, 36], and we believe it could be generalized in future work. We discuss this abstraction, and its limitations further in Section 8.2. Internally, each ToR in the leaf layer has a host that generates demand. The host is connected to the ToR in a single bidirectional connection. Furthermore, each ToR has one bidirectional port towards the spine switch. The spine switch has n input and n output ports of equal capacity, connected via a single directed *matching* between its input and output ports, creating a directed topology. In our model, the spine switch can dynamically change its matching while the network operates via a *reconfiguration*. As mentioned before, the switch implements a single matching at any time. In this matching, any node could be connected to any other node via a single bidirectional connection, but only to one other node. We use the assumption that only single-hop routing is allowed during each matching. However, in our model, to facilitate two-hop routing, data sent to a node in a particular matching can be stored in that node and then forwarded to its destination in a future matching (in a more realistic setting, this can be accomplished by, for example, storing the data in a ToR via the host, and forwarding it in the next matching, as in [14]). This allows us to simulate networks of arbitrary degrees over time. We state that reconfiguration incurs a temporal cost in the form of *reconfiguration time*, or delay, which we will denote as R . Furthermore, we assume that this delay is *global*. That is, when transitioning between two matchings, our model assumes that the entire switch is reconfiguring and that transmission is not possible on all ports, regardless if all ports were changed. The *transmission rate* of all the ports of the switches connected to or from the spine switch in our network is equal and denoted as r , and, w.l.o.g., will be typically set to $r = 1$. We assume that the transmission rate of the host-to-ToR link is infinite. This last assumption allows us to focus on the topology formed between the ToRs. A similar assumption was used in [33].

In this paper, M will denote our $n \times n$ demand matrix in bits of traffic between ToRs. That is $M[i, j]$ is the amount of traffic that ToR (node) T_i needs to send to ToR T_j . Unless otherwise stated, we assume that M is doubly stochastic (i.e., $r = 1$).

2.2. The Topology and the Traffic Schedules

To fully define a dynamic network, we need to define its topology and how traffic is sent on each topology at each configuration index. To do so, we define two types of schedules: a topology schedule $\mathcal{S}_{\text{top}}$ and a traffic schedule $\mathcal{S}_{\text{pkg}}$. In our model, both schedules are created for a certain traffic matrix M , and after the schedule is complete, all the demand in M is finished, and the system may proceed to a different M .

Topology Schedule. A topology schedule $\mathcal{S}_{\text{top}}$ as an ordered set of $v = |\mathcal{S}_{\text{top}}|$ tuples of three elements $\{C_i, \alpha_i, R_i\}$. Where in this context, we denote i as the configuration index. In each tuple, the first element is a permutation matrix $C_i \in P_\pi$ (which can be translated to a switch configuration as a matching). The second element of the tuple is the transmission time $\alpha_i > 0$, the time the system holds active the i 'th switch configuration, C_i , which we also denote as *configuration index i* . The third element is a reconfiguration time $R_i \geq 0$: the time it takes to reconfigure between consecutive switch configurations, i.e., replacing the C_i configuration with the C_{i+1} configuration. No transmission is possible during this time.

We define the *completion time* (denoted as *Time*) of a topology schedule $\mathcal{S}_{\text{top}}$ as the sum of all transmission times α_i plus the reconfiguration times R_i . Formally,

$$\text{Time}(\mathcal{S}_{\text{top}}) = \sum_{i=1}^{|\mathcal{S}_{\text{top}}|} (\alpha_i + R_i) \quad (1)$$

Traffic Schedule.

A traffic schedule defines all traffic sent over a topology schedule, it represents the *amount* of traffic sent between each source and destination node pair. Like the topology schedule, the traffic schedule, $\mathcal{S}_{\text{pkg}}$, is represented by an ordered set of tuples. Since we focus only on schedules that allow paths of at most two hops, each tuple contains *three* matrices representing the network traffic at configuration index i . The *first* matrix denotes traffic *direct* traffic sent over single-hop paths. The *second* and *third* matrices denote *indirect* traffic sent over two-hop paths: one matrix denotes the traffic on the first hop of each two-hop path, and the other matrix denotes the traffic on the second hop. Also note that in our model, we allow a packet to traverse at most a single hop in each configuration C_i . The length of the traffic schedule is the same as the length of the topology schedule, i.e., $v = |\mathcal{S}_{\text{pkg}}|$, as at each configuration index i , a configuration is matched with a tuple from the traffic schedule. For a rigorous and detailed formulation of traffic schedules used in our analysis, we refer the reader to Appendix B. The simplified definition above suffices for the main segment of the paper.

We will shortly describe two further definitions for feasible and complete schedules.

We say that a pair of topology and traffic schedules $(\mathcal{S}_{\text{top}}, \mathcal{S}_{\text{pkg}})$ are *feasible* only if several logical conditions hold, which are necessary for the schedules to be executable. These conditions include, for example, the requirement that second-hop packets can be sent only after the first-hop was sent. We list these conditions formally in the appendix at Section B.1, and for now, consider only feasible schedules. Lastly, we would like our traffic schedules to send all the demand provided. Informally, We define a pair of feasible $\mathcal{S}_{\text{top}}$ and $\mathcal{S}_{\text{pkg}}$ schedules as *complete schedules* if they satisfy all demand in M .

2.3. Problem Definition: The DCT and Throughput of a System

This section defines the main metric of interest studied in this work. In the spirit of previous works, we are interested in the *throughput* of different network designs, or as we denote them here, *systems*. We use

the *demand completion time* (DCT) explained below as a proxy to study the throughput. While previous analytical works studied separately topology and traffic, i.e., either systems with given static topologies [37], or dynamic topologies with naive traffic scheduling [18, 27], or only one specific kind of system [15], we extend the model to support the integration of *both* topology and traffic schedules. To the best of our knowledge, this is the first work that formalizes the joint optimization of both the topology and the traffic scheduling, enabling us to compare different systems: dynamic, static, demand-aware, or demand-oblivious, and with different parameters, on a simple model.

We take the worst-case approach of a classical *throughput* definition by Jyothi et al. [33] and several followups [16, 37, 15] and define the *demand completion time* (and accordingly the throughput) of a system, as a function of the *worst-case* demand matrix for that system. Following this definition, and considering, for example, all double stochastic matrices, we can say that a system that has a *system DCT* of x seconds can serve *any* demand matrix in less or equal x seconds. In turn, we can compare different systems by their system DCT (or throughput).

A system SYS is defined as the set of all possible pairs of schedules $(\mathcal{S}_{\text{top}}, \mathcal{S}_{\text{pkg}})$ that it supports. To give some examples, a system could be defined by all schedules with a specific reconfiguration time resulting from the system hardware, all schedules that use only single-hop routing, all schedules that have a demand-oblivious topology, or all schedules that use only a static topology. In the next section, we will formally define several such known systems.

A *topology scheduling algorithm*, $\mathcal{A}_{\text{top}}$, is an algorithm that generates a topology schedule, and a *traffic scheduling algorithm*, $\mathcal{A}_{\text{pkg}}$, is an algorithm that generates a traffic schedule. We say that a pair of algorithms belongs to a system SYS when their generated schedules (for any demand) are in SYS , i.e., $(\mathcal{A}_{\text{top}}, \mathcal{A}_{\text{pkg}}) \in \text{SYS}$. We can now define the different types of completion times, denoted as Δ , starting with the *matrix completion time*.

Definition 1 (Matrix completion time). *For a given demand matrix M and a pair of schedule algorithms $(\mathcal{A}_{\text{top}}, \mathcal{A}_{\text{pkg}})$ that generate two complete schedules $(\mathcal{S}_{\text{top}}, \mathcal{S}_{\text{pkg}})$, the completion time of the matrix M is defined as the time it takes to execute the schedules,*

$$\Delta(\mathcal{A}_{\text{top}}(M), \mathcal{A}_{\text{pkg}}(M), M) = \text{Time}(\mathcal{A}_{\text{top}}(M)).$$

The demand completion time for a *pair of algorithms* is defined as the matrix completion time of their *worst-case* matrix.

Definition 2 (Algorithms completion time). *Let $\mathcal{M}$ be the family of all double stochastic matrices. Let $(\mathcal{A}_{\text{top}}, \mathcal{A}_{\text{pkg}})$ be a pair of schedule algorithms. The algorithms' completion time is defined,*

$$\Delta(\mathcal{A}_{\text{top}}, \mathcal{A}_{\text{pkg}}, \mathcal{M}) = \max_{M \in \mathcal{M}} \Delta(\mathcal{A}_{\text{top}}(M), \mathcal{A}_{\text{pkg}}(M), M).$$

We can now finally define the *demand completion time* of a system SYS , denoted as *system DCT*,

Definition 3 (System DCT). Let $\mathcal{M}$ be the family of all double stochastic matrices. Consider a network design system SYS . The demand completion time of a SYS is defined as the completion time of its best algorithms,

$$\Delta(\text{SYS}, \mathcal{M}) = \min_{(\mathcal{A}_{\text{top}}, \mathcal{A}_{\text{pkg}}) \in \text{SYS}} \Delta(\mathcal{A}_{\text{top}}, \mathcal{A}_{\text{pkg}}, \mathcal{M}).$$

In previous work [16], the throughput, for a particular demand matrix M , was defined as the highest scaling factor $0 \leq \theta(M) \leq 1$ for which the traffic is feasible in the system. Furthermore, the throughput of a system, denoted as θ^* , is the worst-case throughput among all demand matrices M .

Furthermore, as noted in [16], the DCT has a direct relation to the *throughput* of the system. In particular, it follows that the throughput of a system SYS , defined in bits per second, is the reciprocal of the system DCT, $\Delta(\text{SYS})$. For complete schedules and double stochastic matrices, we then define

$$\theta^* = \text{Throughput}(\text{SYS}, \mathcal{M}) = \frac{1}{\Delta(\text{SYS}, \mathcal{M})}, \quad (2)$$

where, θ^* is as defined in [16]. However, in the context of this paper, the matrix family is not assumed to be all-inclusive, and the system and the matrix family are denoted explicitly.

In the rest of the paper, we are set to study the system DCT of different systems.

We define three major systems, BVN-SYS, RR-SYS, and COMP-SYS, and explore their properties and tradeoffs in the journey to better understand the DCT and throughput of different systems with dynamic topologies. We show that our model from Section 2 fits known systems like BVN-SYS and RR-SYS, enabling us to study their DCT and derive familiar and new results, in particular, we use these results in our later analysis on COMP-SYS. Recall that our demand is represented by an $n \times n$ saturated doubly stochastic matrix M , which means that $\|M\|_{\Sigma} = nr$ and $r = 1$. However, when appropriate, we write these parameters explicitly so the bounds and results are generalized to *scaled* doubly stochastic matrices that are not necessarily saturated, i.e., $\|M\|_{\Sigma} < nr$. In our analysis of the system DCT, we assume that the delay is only due to switch reconfiguration and transmission times. We neglect other factors, such as packet loss and congestion, and focus on fundamental tradeoffs. In this paper, we study a subset of double stochastic matrices, denoted as $M(v)$ and the $M(v, u)$ family of matrices. As we will see, these capture the edge cases for both BVN-SYS and RR-SYS. We, therefore, believe their exploration gives interesting insights into the general case. We define them in the following section.

2.4. The $M(v)$ and the $M(v, u)$ Matrices.

In the paper, we use the following matrices: Let $M(v)$ be defined as a set of double stochastic matrices where each row and column has v non-zero, non-diagonal elements, each equal to $\frac{1}{v}$. Similarly, let $M(v, u) = uM(n-1) + (1-u)M(v)$ where $0 \leq u \leq 1$. We note that any $M \in M(v, u)$ is doubly stochastic with permutations (i.e., $M(1)$) and the uniform matrix (i.e., $M(n-1)$) as special cases. We believe this family of matrices can capture some of the complexities of real network traffic in a more concise framework and is similar in spirit to the traffic models used in [18, 20, 27], and others.

3. The Demand-Aware Topology System: BvN-sys

We first consider in our formal model the demand completion times of the well-established system, BvN-sys. The BvN-sys system is *demand-aware*, meaning that the topology schedule is computed based on a decomposition of the demand matrix M . The traffic schedule of the BvN-sys, in turn, is very simple and supports only single-hop paths. Examples of networks with this type of system are FireFly [21], Eclipse [18], Solstice [20], and ProjecTor [17].

Recall that the topology schedule $\mathcal{S}_{\text{top}}$ is a list of matchings C_i and coefficients α_i . In BvN-sys we assume that $\mathcal{S}_{\text{top}}$ is generated through a BvN decomposition (See Appendix at [Appendix A](#) for background), and for each i in the decomposition, we set the switch configuration $C_i = P_i$. The specific BvN algorithm can vary, but the goal is to minimize the number of reconfigurations (in the simulation, we use the state-of-the-art algorithm [27]). Since $\mathcal{S}_{\text{top}}$ is based on a BvN decomposition and M is doubly stochastic, the following will hold, $\sum_{i=1}^{|\mathcal{S}_{\text{top}}|} \beta_i C_i = M$ and the sum of coefficients is one, that is, $\sum_{i=1} \beta_i = 1$. The transmission time is then $\alpha_i = \beta_i / r$. In BvN-sys and its topology schedules, we assume that the reconfiguration time between matchings is a constant, denoted R_b , and is identical to all times, i.e., $\forall i, R_i = R_b$. The reconfiguration time for BvN systems is considered *high*, in particular in comparison to demand-oblivious topology schedules. This is assumed for two reasons: i) either the flexibility that the topology requires (enabling any possible matching) results in a more significant time to set the configuration and/or ii) the time to *compute* the next matching (usually in some greedy way) requires much more time than in a precalculated matching.

In BvN-sys, both the topology and the traffic schedules are of specific types. Let $\mathcal{A}_{\text{top}} = \text{BvN}$ be an algorithm that decomposes M into $v = |\mathcal{S}_{\text{top}}|$ matching (i.e., permutation matrices) and generates the topology schedule. The algorithm that generates the traffic schedule that sends only direct, single-hop traffic is denoted as $\mathcal{A}_{\text{pkg}} = \text{DIRECT}$. We can define BvN-sys formally in the following manner.

Definition 4 (The BvN-System, BvN-sys). *BvN-sys is defined as a system that must use a topology schedule BvN.*

Following Eq. (1) we get that for a double stochastic matrix M and $r = 1$ the *matrix* completion time is

$$\Delta(\text{BvN}, \text{DIRECT}, M) = \sum_{i=1}^v (\alpha_i + R_b) = 1 + vR_b \quad (3)$$

We note that the matrix DCT is linear in the number of reconfigurations v . It will be minimal when the number of reconfigurations is one, namely a single permutation matrix. Regarding the worst-case matrix, the BvN decomposition of any *positive* demand matrix (i.e., where all elements other the diagonal are positive) requires at least $n - 1$ permutations and so $n - 1$ reconfigurations to be fully transmitted, one such example is the *uniform* matrix. We can, therefore, state the following observation on the system DCT of BvN-sys.

Observation 1 (The system DCT of BvN-sys). *The system DCT of BvN-sys for $r = 1$ is at least*

$$\Delta(\text{BvN-sys}, \mathcal{M}) \geq 1 + (n - 1)R_b \quad (4)$$

4. The Demand-Oblivious System: RR-SYS

This system is based on a *demand-oblivious* topology and is denoted as the round-robin systems, RR-SYS. The RR-SYS system is motivated by recent proposals for DCN design like RotorNet [14], Opera [22], and Sirius [12]. These types of networks are also denoted as 'rotor'-based designs.

In RR-SYS the system topology is composed of a pre-determined and pre-computed set of $n - 1$ matchings (i.e., permutation matrices, switch configurations) whose union forms a *complete graph*. Let $\mathcal{C} = \{C^1, \dots, C^{n-1}\}$ be the set of switch configurations denoted as *cycle configuration*, where each C^i is a permutation matrix. The topology scheduler $\mathcal{S}_{\text{top}}$ transitions from one switch configuration in $\mathcal{C}$ to another in a periodic, round-robin, and fixed manner, oblivious to the demand. The period of $n - 1$ switch configurations that emulate the complete graph is called a *cycle*. Each configuration C^i is held for a time δ , i.e., $\forall i, \alpha_i = \delta$. The reconfiguration time in the topology schedule is also constant and denoted as R_r , $\forall i, R_i = R_r$. An important property of the RR-SYS topology schedule is the *duty cycle* as defined in prior work [14, 22] to be η where $\eta = \frac{\delta}{\delta + R_r}$. This is the fraction of time spent sending data, δ , to the total time spent in a slot, $\delta + R_r$. We denote the algorithm that generates such a round-robin topology schedule as $\mathcal{A}_{\text{top}} = \text{RR}$. We define RR-SYS formally in the following manner.

Definition 5 (The Round-Robin System, RR-SYS). *RR-SYS is defined as a system that must use RR as its topology schedule.*

We discuss next the set of traffic schedulers that RR-SYS may choose from. Up-to-date, there is no single, optimal, well-accepted traffic scheduling algorithm for RR-SYS. Previous work [12] considered 2-hops Valiant routing [29]. While this approach uses oblivious routing and is easy to implement, it is not optimal regarding worst-case demand completion times for RR-SYS. A different example, RotorNet, uses RotorLB [14], a decentralized protocol that uses adaptive routing and allows nodes to negotiate routing decisions. Briefly, in RotorLB, a single hop is preferred over two hops, and nodes only help other nodes if they have available capacity, thus spreading the load more evenly. The bounds of this approach are not clear. Nevertheless, here we present several algorithms and bounds that enable us to find the system DCT of RR-SYS formally.

4.1. The system DCT of RR-SYS

Recall that in our model, traffic can be sent *directly* (in a single hop) or *indirectly* (using two hops). A traffic scheduling algorithm for RR-SYS should decide which traffic in the demand matrix M is routed directly using a single hop and which traffic is routed indirectly in two hops on the RR-SYS topology schedule RR. Our first result is a lower bound for the system DCT of RR-SYS, by a lower bound on the completion time of a permutation demand matrix for any traffic scheduling algorithm.

Theorem 1. *[Lower bound for RR-SYS system DCT] Let $\mathcal{A}_{\text{pkg}}$ be any traffic scheduling algorithm for RR-SYS that generates a complete schedule. Let $P \in P_\pi$ be any derangement permutation matrix, then we have*

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, P) \geq (2 - \frac{2}{n}) \frac{1}{\eta r} \frac{\|P\|_\Sigma}{n} \quad (5)$$

We provide details of the proof in the appendix in [Section C.1](#). However, for brevity, we provide the following proof overview.

Proof Overview for Theorem 1. Our proof has two main parts. We first prove that the lower bound is a function of the skew parameter we denote as ϕ , then prove an upper bound on this parameter. The skew parameter, ϕ , denotes the fraction of traffic sent via a single hop (i.e., a path length of one) in a given traffic schedule, and $1 - \phi$ is the fraction that is sent on two hops (i.e., a path length of two). It follows that $(2 - \phi)$ is the average path length of a given schedule. We give a formal definition for ϕ in the appendix using [Definition 13](#). Following our definition, In [Theorem 12](#), we show that the DCT of RR-SYS is lower bounded by $(2 - \phi) \frac{1}{\eta r} \frac{\|M\|_\Sigma}{n}$.

To find a lower bound on the DCT, we need to increase ϕ (and decrease the average path length). We, therefore, in [Theorem 13](#), prove the upper bound for ϕ with any optimal traffic scheduler $\mathcal{A}_{\text{pkg}}$ on any demand permutation matrix in $M(v)$. As part of the proof of our theorem, we state the following corollary of [Theorem 12](#) and [Theorem 13](#).

Corollary 2 (Lower bound for $M(v)$ on RR-SYS). *Let $\mathcal{A}_{\text{pkg}}$ be any traffic scheduling algorithm for RR-SYS that generates a complete schedule. Given an $n \times n$ matrix $M \in M(v)$, the lower bound for the DCT of RR-SYS is*

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, M(v)) \geq (2 - \frac{2v}{n-1+v}) \frac{1}{\eta r} \frac{\|M\|_\Sigma}{n} \quad (6)$$

This results from combining the lower bound on the DCT of RR-SYS with the upper bound on ϕ for matrices from $M(v)$. Finally, we note that the worst case is for $M(1)$, i.e., $v = 1$, that represents the case of permutation matrix $P \in P_\pi$, and imply $\phi = \frac{2}{n}$ which proves [Theorem 1](#).

Next, we consider a traffic scheduler, which, surprisingly, achieves the lower bound we stated before for any derangement permutation matrix $P \in P_\pi$ on RR-SYS. We denote this as the permutation scheduler PERM. Briefly, PERM, generates a schedule which equally distributes the demand from the permutation demand cells such that each connection will eventually send a total of $\frac{2}{n}$ (normalized) bits. We provide a full description of this algorithm in the appendix in [Section C.2](#). This includes its pseudo-code in [Algorithm 2](#) and a proof for the following lemma.

Lemma 3. *[DCT for the PERM Algorithm] Let $P \in P_\pi$ be an $n \times n$ scaled permutation matrix. Then, the schedule generated from the algorithm $\mathcal{A}_{\text{pkg}} = \text{PERM}$, is feasible, complete, and has a demand completion time, Δ of,*

$$\Delta(\text{RR}, \text{PERM}, P) = (2 - \frac{2}{n}) \frac{1}{\eta r} \frac{\|P\|_\Sigma}{n} \quad (7)$$

However, PERM only works for a single permutation. To find the system DCT, we need an algorithm for a general double stochastic matrix. Next, consider an algorithm that builds a schedule using *multiple permutations*. We denote this scheduler as MULP. This algorithm works in the following way: It decomposes any double stochastic demand matrix M into a set of *permutation matrices* (regardless of how many) and sends each using the permutation scheduler, PERM. The pseudo-code for this algorithm appears in [Algorithm 3](#). Formally, we prove the following result about the DCT in the RR-SYS system, which matches the lower bound of [Theorem 1](#).

Theorem 4. [*Upper bound for RR-SYS system DCT*] *Let M be a scaled doubly stochastic matrix. Then, using the traffic scheduling algorithm, MULP the matrix demand completion time in RR-SYS is*

$$\Delta(\text{RR}, \text{MULP}, M) = (2 - \frac{2}{n}) \frac{1}{\eta r} \frac{\|M\|_{\Sigma}}{n}. \quad (8)$$

Proof. Let M be a λ scaled doubly stochastic matrix. Using BvN, we can decompose a scaled double stochastic matrix to v different matchings. We get a list of v coefficients that sum to λ , that is, $\sum_{i=1}^v \beta_i = \lambda$, and a list of v permutation matrices P_i . We can then schedule each matching separately on RR-SYS using the algorithm PERM, as shown in [Lemma 3](#). Recall that the sum of the product of the coefficients β_i and permutation matrices P_i is equal to the original traffic matrix, i.e., $\sum_{i=1}^v \beta_i P_i = M$. It also holds for the sum of the matrix weights $\sum_{i=1}^v \|\beta_i P_i\|_{\Sigma} = \|M\|_{\Sigma}$. The DCT of MULP is the sum of the DCTs of all the different permutations, that is

$$\begin{aligned} \Delta(\text{RR}, \text{MULP}, M) &= \sum_{i=1}^v \Delta(\text{RR}, \text{PERM}, \beta_i P_i) = \sum_{i=1}^v (2 - \frac{2}{n}) \frac{1}{\eta r} \frac{\|\beta_i P_i\|_{\Sigma}}{n} \\ &= (2 - \frac{2}{n}) \frac{1}{\eta r n} \sum_{i=1}^v \|\beta_i P_i\|_{\Sigma} = (2 - \frac{2}{n}) \frac{1}{\eta r} \frac{\|M\|_{\Sigma}}{n} \end{aligned}$$

We would note that the length of each schedule changes. We, therefore, need to change the slot hold time, δ , accordingly for each sub-scheduled permutation matrix so that the schedule finishes at the correct time. Since the sum of all schedules achieves the value stated in the theorem, BvN will work for any doubly stochastic matrix, and the theorem follows. $\square$

We note that our proof and MULP algorithm assumes that RR-SYS can use a dynamic value for δ . This is very useful for the purpose of our proof and bounds; however, it might not be practically possible. Another option is to use a very small value for δ , (which is the case in related works [\[14, 22\]](#)). This would mean that RR-SYS might spend a short time idle and not transmitting at the end of the schedule. We leave the details of the practical implementation for future work.

While it was previously shown that permutation matrices are the worst case for RR-SYS type systems, both formally [\[16, 28\]](#) and informally [\[14, 12\]](#), the above theorem closes a small gap and enables us to state the system DCT of RR-SYS formally. To summarize our RR-SYS analysis, we offer the following theorem for the system DCT of RR-SYS, which follows directly from [Theorem 4](#).

Theorem 5 (The system DCT of RR-SYS). *Let $\mathcal{M}$ be the set of all double stochastic matrices. For $r = 1$, and $R_r = 0$, the system DCT of RR-SYS is:*

$$\Delta(\text{RR-SYS}, \mathcal{M}) = 2 - \frac{2}{n}. \quad (9)$$

Proof. [Theorem 4](#) proves that the bound stated in the theorem is feasible using the MULP algorithm for any demand matrix. [Theorem 12](#) proves that there is a matrix that takes at least the bound in the theorem. This proves that the above bound is the best-case DCT for worst-case demand. $\square$

While we have shown an algorithm that achieves the system DCT for any double stochastic matrix, we can still improve the DCT of RR-SYS for some particular matrices. To see how, we first define a simple traffic schedule algorithm, $\mathcal{A}_{\text{pkg}} = \text{DIRECT}$. This algorithm creates a schedule by sending packets exclusively using a single hop. The following DCT bound was shown in previous work [\[16\]](#), we repeat it here. We further give a proof in the appendix in [Section C.4](#) to demonstrate that it is aligned with our definitions and model.

Theorem 6. *[DIRECT upper bound] Let M be any demand matrix and DIRECT be a traffic schedule where all packets are sent over a single hop. The matrix completion time in RR-SYS is then*

$$\Delta(\text{RR}, \text{DIRECT}, M) = (n-1) \frac{\max(M_{i,j})}{\eta r} \quad (10)$$

For the worst-case demand matrix, the permutation matrix, this algorithm has a much worse DCT than MULP. However, it will improve the DCT significantly when faced, for example, with a near uniform or at best, a uniform matrix. When $r = 1$, $R_r = 0$ the matrix completion time is optimal. More formally;

Observation 2. *[DIRECT DCT for uniform traffic] Let $M(n-1)$ be a uniform demand matrix. For $r = 1$, and $R_r = 0$, the matrix completion time for RR-SYS using DIRECT is optimal that is*

$$\Delta(\text{RR}, \text{DIRECT}, M(n-1)) = (n-1) \frac{1}{n-1} = 1 \quad (11)$$

This follows from each cell in $M(n-1)$ having a load of $n-1$. The question of the best traffic schedule for RR-SYS on any particular matrix is still open. Therefore, in the rest of the work, we consider a simple algorithm for RR-SYS that combines the best results between [Theorem 6](#) and [Theorem 4](#). Since both MULP and the naive DIRECT traffic scheduling algorithms are defined on the general doubly stochastic matrix, we can combine them to form an improved algorithm UPPER. This algorithm takes the best traffic schedule between the two, depending on the traffic matrix M that is,

Definition 6 (UPPER algorithm for RR-SYS DCT).

$$\Delta(\text{RR}, \text{UPPER}, M) = \min(\Delta(\text{RR}, \text{MULP}, M), \Delta(\text{RR}, \text{DIRECT}, M))$$

Using this definition and comparing the expressions for DCT from [Theorem 4](#) and [Theorem 6](#), we can arrive at the following observation. When the maximal element in M is larger than nearly twice (i.e., by $(2 - \frac{2}{n})$ the average matrix value $\frac{\|M\|_{\Sigma}}{n(n-1)}$) we will use the bound from [Theorem 4](#) and otherwise, the bound from [Theorem 6](#),

$$\Delta(\text{RR}, \text{UPPER}, M) = \begin{cases} (2 - \frac{2}{n}) \frac{1}{\eta r} \frac{\|M\|_{\Sigma}}{n}, & (2 - \frac{2}{n}) \frac{\|M\|_{\Sigma}}{(n-1)n} \leq \max(M_{i,j}) \\ \frac{(n-1) \max(M_{i,j})}{\eta r}, & \text{otherwise} \end{cases} \quad (12)$$

We see that, indeed, UPPER cannot improve the result of MULP in the worst case.

In the next section, we consider a new alternative to the two existing systems we discussed, COMP-SYS, and study its performance.

Algorithm 1 The PIVOT Algorithm

Require: $M \in \mathbf{R}^{+^{n \times n}}$ scaled double stochastic

Ensure: $M^{\text{BvN}}, M^{\text{RR}}$

- 1: $P_M = \{P_1, \dots, P_v\}, \beta_M = \{\beta_1, \dots, \beta_v\}$ ▷ Decompose M to a set permutation matrices and coefficients using BvN, sorted by β_i from large to small
 - 2: $D_0 = \Delta(\text{RR}, \text{UPPER}, M), D_v = \Delta(\text{BvN}, \text{DIRECT}, M)$
 - 3: **for** $i \in [1, v - 1]$ **do**
 - 4: $M^{\text{BvN}} = \sum_{j=1}^i \beta_j P_j$
 - 5: $M^{\text{RR}} = \sum_{j=i+1}^v \beta_j P_j$
 - 6: $D_i = \Delta(\text{BvN}, \text{DIRECT}, M^{\text{BvN}}) + \Delta(\text{RR}, \text{UPPER}, M^{\text{RR}})$
 - 7: **end for**
 - 8: $f = \arg \min_i D_i$ ▷ Find best decomposition
 - 9: $M^{\text{BvN}} = \sum_{j=1}^f \beta_j P_j$ and $M^{\text{RR}} = \sum_{j=f+1}^v \beta_j P_j$
-

5. The Composite System and The Pivot Algorithm

Before introducing COMP-SYS, we make a simple observation following the previous sections that will serve as our main motivation. The *worst-case* demand matrix for RR-SYS (i.e., a permutation matrix) is the *best-case* demand matrix for BvN-SYS. The opposite is also almost the same: the *best-case* demand matrix for RR-SYS (i.e., a uniform matrix) is a very bad case for BvN-SYS (but not the worst). This motivates COMP-SYS, which is a system that can adapt to different types of demand using advantages from both the BvN-SYS and RR-SYS systems combined. While both previously discussed systems, RR-SYS and BvN-SYS, were able to use a *single* type of scheduler, we define COMP-SYS as a system and a network that can use both demand-aware and demand-oblivious types of topology schedules for the *same* demand matrix. Additionally, it must adjust its traffic schedule to make it feasible and complete.

One way to implement a schedule that uses demand-aware and demand-oblivious topologies is to decompose the demand matrix into two parts and use different schedule pairs for each part. We will define such an algorithm for COMP-SYS in this section. But first, formally, we define COMP-SYS as the following system.

Definition 7 (The Composite System, COMP-SYS). *Let M be any double stochastic demand matrix. COMP-SYS is defined as a system that decomposes M , into two scaled double stochastic matrices M^{BvN} and M^{RR} , such that $M = M^{\text{BvN}} + M^{\text{RR}}$. It then schedules the topology and traffic for M^{BvN} with BvN-SYS and sequentially for M^{RR} using RR-SYS.*

Next, we describe a novel decomposition algorithm that we will use for our evaluation of COMP-SYS, both for the empirical and theoretical analysis. Algorithm PIVOT (see Algorithm 1 for the pseudo-code), starts with BvN matrix decomposition of the demand matrix M into v permutation matrices and coefficients. W.l.o.g., we assume that all permutations P_i are sorted by their weight, that is, by their coefficients β_i , where β_1 is the largest coefficient. Essentially, PIVOT is searching for the best partition of the permutations. It creates $v + 1$ partitions of the decomposition of M to M^{BvN} and M^{RR} . Each partition is defined by the index of a *pivot permutation* where all permutations strictly heavier than the pivot permutation create

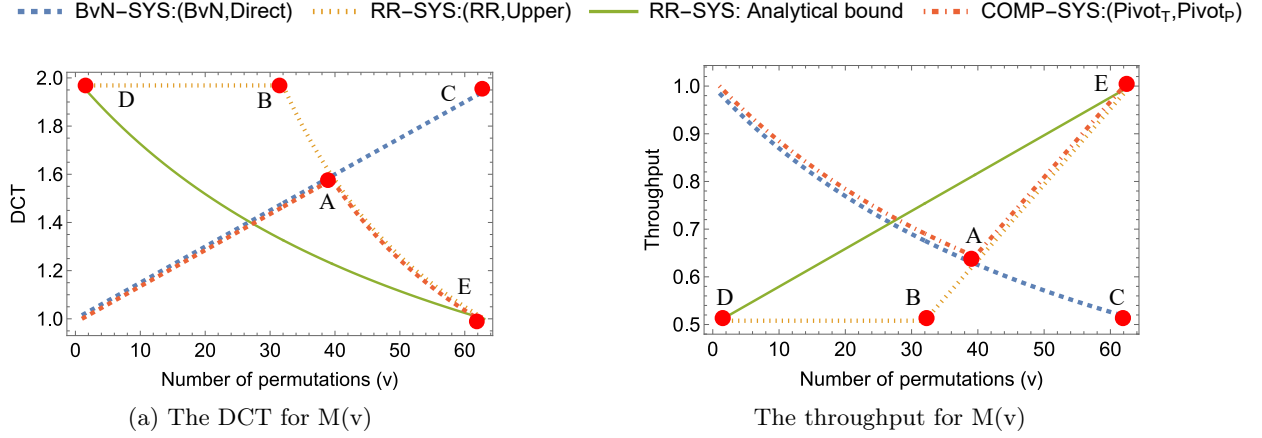


Figure 2: The demand completion time and throughput for the Family of $M(v) = M(v, 0)$ matrices.

M^{BvN} and all smaller create M^{RR} . We note that the extra partition (one more than V) is tested since PIVOT also examines the cases where $M^{\text{BvN}} = M$ and $M^{\text{RR}} = M$, that is, where all permutations are sent using just one system (Line 2). By comparing the expected DCT D_i of each partition i (Line 9), PIVOT returns the decomposition of M into M^{BvN} and M^{RR} , that minimizes the DCT.

Let PIVOT_T and PIVOT_P denote the topology and traffic schedules resulting for the PIVOT algorithm, respectively. Formally, PIVOT_T is the concatenation of the topology schedules $\text{BvN}(M^{\text{BvN}})$ and $\text{RR}(M^{\text{RR}})$, and PIVOT_P is the concatenation of the traffic schedules $\text{DIRECT}(M^{\text{BvN}})$ and $\text{UPPER}(M^{\text{RR}})$.

We can state the following general theorem for COMP-SYS and PIVOT

Theorem 7 (DCT of COMP-SYS with PIVOT). *For any scaled doubly stochastic matrix M , the DCT of COMP-SYS is at most as the lowest DCT between RR-SYS or BvN-SYS,*

$$\Delta(\text{PIVOT}_T, \text{PIVOT}_P, M) \leq \min(\Delta(\text{BvN}, \text{DIRECT}, M), \Delta(\text{RR}, \text{UPPER}, M)) \quad (13)$$

Proof. This follows since PIVOT tests and also compares the cases $(M^{\text{BvN}} = M, M^{\text{RR}} = 0_n)$, and $(M^{\text{BvN}} = 0_n, M^{\text{RR}} = M)$. Therefore, PIVOT will be at least as good as either. $\square$

We discuss the running time of PIVOT in the appendix [Appendix D](#). But interestingly, we emphasize that PIVOT only needs to make a single BvN decomposition during its execution.

As for the previous systems, we would like to find the system DCT for COMP-SYS, namely, to find the worst-case demand matrix for it and its completion time.

In the next section, we make a step in this direction, and we study the DCT of COMP-SYS and PIVOT and compare it to BvN-SYS and RR-SYS for the special case of the $M(v, u)$ family of matrices.

6. A Case Study: The system DCT of COMP-SYS for the $M(v, u)$ Matrices Family

In [Section 3](#), and [Section 4](#), we discussed the bounds and worst-case DCT of the systems RR-SYS and BvN-SYS for a general double stochastic matrix. However, no such bounds are known for COMP-SYS type

systems for this general matrix. That is, both the worst-case matrix and worst-case DCT value are unknown. This section will discuss the DCT bounds of RR-SYS and BVN-SYS compared to our COMP-SYS system for a wide family of double stochastic traffic matrices $M(v, u)$. We leave the general question open for future research. Recall that we first defined the $M(v, u)$ matrix family in [Section 2.4](#).

Throughout this evaluation, we will use several assumptions: $\eta = 1$, which means that the reconfiguration time $R_r = 0$. The transmission rate is normalized, meaning that the transmission rate is $r = 1$. We set $R_b = \frac{1}{n}$ for the numerical examples and demonstrations. As we will see later, using this value means that both RR-SYS and BVN-SYS have the same system DCT. This ensures no system has an obvious advantage over the other on any matrix from the set $M(v, u)$. Furthermore, a similar assumption is made in Eclipse [\[18\]](#). Lastly, since in this section, we assume that $n = 64$, we get $R_b = \frac{1}{64} \approx 15ms = 0.015s$. We note that this value for R_b is also a good approximation of the actual value of dynamic switch reconfiguration times [\[38, 11\]](#).

Let us now consider the main result of this section, the system DCT for each of our systems, in the following theorem. This theorem describes the system DCT of all of our three systems of interest on the matrix family $\mathcal{M} = M(v, u)$.

Theorem 8 (Systems DCT for $M(v, u)$). *Let $\mathcal{M}$ be the family of all $M(v, u)$ matrices. Then,*

- *The system DCT of BVN-SYS is:*

$$\Delta(\text{BVN}, \mathcal{M}) = 1 + R_b(n - 1) \quad (14)$$

which is about 1.94 for $n = 64$ and $R_b = \frac{1}{n}$. The worst-case is, for example, the matrix is $M(n - 1, 0)$.

- *The system DCT of RR-SYS is:*

$$\Delta(\text{RR}, \mathcal{M}) = 2 - \frac{2}{n} \quad (15)$$

which is about 1.96 for $n = 64$. The worst-case matrix is $M(1, 0)$, a permutation matrix.

- *The system DCT of COMP-SYS for a BVN reconfiguration time $R_b < \frac{(2n-4)}{n^2}$:*

$$\Delta(\text{COMP-SYS}, \mathcal{M}) \leq \frac{\sqrt{1 + 4R_b(n - 1)} + 1}{2} \quad (16)$$

which is about 1.6 for $n = 64$ and $R_b = \frac{1}{n}$. The worst-case matrix is $M(\ddot{v}, 0)$, where $\ddot{v} = \frac{\sqrt{1 + 4R_b(n - 1)} - 1}{2R_b}$.

For all systems the worst-case matrices are achieved for $u = 0$, so $M(v, 0) = M(v)$

Before diving into this theorem, let us first make two important observations. Consider how will PIVOT work with the $M(v)$ family of matrices. When PIVOT algorithm is used with $M(v)$, it converges into a simpler form. Since, in this case, there is only one β_i size (as seen in the pseudo-code in [Algorithm 1](#)), we will always send the entire matrix using either BVN-SYS or RR-SYS. The DCT of this system and algorithm is the following observation for the $M(v)$ family.

Observation 3 (DCT of COMP-SYS with PIVOT on $M(v)$).

$$\Delta(\text{PIVOT}_T, \text{PIVOT}_P, M(v)) = \min(\Delta(\text{BVN}, \text{DIRECT}, M(v)), \Delta(\text{RR}, \text{UPPER}, M(v))) \quad (17)$$

This follows from the linearity of the DCT in BVN-SYS, and the fact that all permutations are of the same size.

The second observation is the main takeaway of this section. The system DCT COMP-SYS is strictly lower than BVN-SYS and RR-SYS.

Observation 4. *Let $\mathcal{M}$ be the family of all $M(v, u)$ matrices. When $0 < R_b < \frac{(2n-4)}{n^2}$, then the system DCT of COMP-SYS is strictly lower than both BVN-SYS and RR-SYS.*

$$\Delta(\text{COMP-SYS}, \mathcal{M}) < \min(\Delta(\text{BVN}, \mathcal{M}), \Delta(\text{RR}, \mathcal{M})) \quad (18)$$

Part of [Theorem 8](#) claims the worst case matrix for every system is found at $u = 0$, to get an intuition of why this is the case for systems using RR-SYS (RR-SYS and COMP-SYS), we note that we can separate the $M(v, u)$ matrix into its two components, and schedule the uniform component using DIRECT separately and the non-uniform component using either UPPER for RR-SYS or PIVOT for COMP-SYS. Since the uniform matrix $M(n-1)$ can be scheduled at an optimal rate using DIRECT (as in [Observation 2](#)). For BVN-SYS, we can still decompose $M(v, u)$ using $n-1$ matchings, thus, the system DCT is unaffected. We give a more formal proof of this point in the appendix in [Section E.1](#).

An Illustrative Example. To get a glimpse of the results of [Theorem 8](#), we present them for the case of $M(v, 0) = M(v)$ in [Figure 2](#). In this figure, we visualize the general curves of the DCT of our three systems. We present the behavior of the matrix DCT for the entire range of $M(v)$, i.e., $v = 1, \dots, 63$. In [Figure 2 \(a\)](#), we plot the DCT for BVN-SYS (using BVN and DIRECT), RR-SYS (using RR and UPPER), and COMP-SYS (using PIVOT). We additionally plot the lower bound for RR-SYS from [Theorem 12](#), since the optimal DCT for each case is not known. In [Figure 2 \(b\)](#), we see the throughput results of each system. We add five significant points, $[A, B, C, D, E]$, to the figures and discuss them next. Note that the points are homologous to both figures. Looking at the figures, recall that COMP-SYS is using the PIVOT as in [Observation 3](#), thus, the DCT of COMP-SYS can be described as the line representing the minimum between the DCT of the UPPER algorithm in RR-SYS and the DCT of BVN-SYS using BVN, where the two lines meet at point (A). This point is the worst-case demand for COMP-SYS as seen in [Equation 16](#). For RR-SYS, the worst-case matrix is at point (D), which is the permutation matrix as proven in [Theorem 1](#) and [Lemma 3](#) and stated in [Theorem 8](#) and [Equation 15](#). We note that for matrices between points (D) and (B), UPPER has the same bound as in (D). Finally, for BVN-SYS the worst case point is at (C), which, as we noted in [Equation 14](#) is for the uniform matrix where $v = n-1$. Note that the numerical results stated in [Theorem 8](#) fit the location of (A), (B), and (C) in this figure. Looking at the RR-SYS lower bound, we see that it merges with the RR-SYS upper bound using UPPER in both of the extreme points, including the worst-case point at $M(1)$, marked by (D), and best case matrix which is $M(n-1)$ at point (E). This proves that (D) is the

worst matrix for RR-SYS. To conclude, the throughput (i.e., $\frac{1}{\Delta}$) of both BvN-SYS and RR-SYS is about 0.5. By combining the strengths of both systems the throughput of COMP-SYS is 25% higher, at 0.625, as seen in Figure 2 (b). Recall that, regarding to RR-SYS and BvN-SYS, we have already proven the system DCT for general double stochastic matrix, in Theorem 5 for RR-SYS, and in Observation 1. Since the worst-case matrix for each system is part of the $M(v, u)$ matrix family, we focus on developing the results of Theorem Theorem 8 for COMP-SYS. For clarity, we consider the case $u = 0$.

6.1. DCT of COMP-SYS on $M(v)$

In this section, we discuss the DCT of COMP-SYS the $M(v)$ type matrices. Since our definition for COMP-SYS includes the use of a two-component system, BvN and RR-SYS using UPPER algorithm, we first give exact expressions for the DCT of those systems on $M(v)$ and conclude with the DCT of COMP-SYS. This result, in turn, will help to compute the system DCT of COMP-SYS.

The DCT of BvN-SYS on $M(v)$. For BvN-SYS, the result is very straightforward and based directly on previous results. We can use the DCT from Equation 3. We decompose the matrix $M(v)$ into v permutation matrices. And since $\frac{\|M(v)\|_{\Sigma}}{rn} = 1$, we can arrive at the result. $\Delta(\text{BvN}, \text{DIRECT}, M(v)) = 1 + R_b v$.

The DCT of RR-SYS for $M(v)$. We state the DCT on $M(v)$ with RR-SYS in the following lemma.

Lemma 9 (The DCT of RR-SYS on $M(v)$). *The DCT of RR-SYS on the $M(v)$ matrix type is*

$$\Delta(\text{RR}, \text{UPPER}, M(v)) = \begin{cases} (2 - \frac{2}{n}) & \text{for } 1 \leq v < \frac{2}{n} \\ \frac{(n-1)}{v} & \text{otherwise} \end{cases} \quad (19)$$

Proof. Recall the result for the DCT of RR-SYS using algorithm UPPER in Definition 6. This is the minimal value between the bound found in Theorem 4 and the bound found in Theorem 6. Setting the correct values for our setup in these two theorems, i.e., $\eta = 1$, $r = 1$, $\frac{\|M(v)\|_{\Sigma}}{n} = 1$ and $\max(M(v)_{i,j}) = \frac{1}{v}$. We get two monotonous functions. This means we must find the (single) value of v where they intersect. This is $\frac{(n-1)}{v} = (2 - \frac{2}{n})$. Solving for v we get $v \geq \frac{n}{2}$. The Lemma follows. $\square$

We can now continue to the main result of this section, the analysis of the system DCT of COMP-SYS.

The DCT of COMP-SYS for $M(v)$. First, we note that the general expression for the DCT of COMP-SYS is a two-part expression depending on the BvN reconfiguration time R_b . However, in this section, we only present the part relevant to our assumptions, that is, for $R_b < \frac{(2n-4)}{n^2}$ and we note that $R_b = \frac{1}{n} = 0.015$ is in this range when $n = 64$. Furthermore, while we have set $R_r = 0$, for BvN-SYS, if $R_b = 0$, the DCT will be 1 for all matrices, as this is the only 'tax' or cost of BvN-SYS, making this system optimal for any demand. Here we present the case $R_b < \frac{(2n-4)}{n^2}$, the fully general case, including $R_b \geq \frac{(2n-4)}{n^2}$ is in the appendix in Theorem 16.

Lemma 10 (DCT of COMP-SYS on $M(v)$). *For $u = 0$ and $R_b < \frac{(2n-4)}{n^2}$ the DCT of COMP-SYS for $M(v)$ is the following expression.*

$$\Delta(\text{PIVOT}_T, \text{PIVOT}_P, M(v)) = \begin{cases} 1 + R_b v & 1 \leq v < \frac{\sqrt{1+4R_b(n-1)}-1}{2R_b} \\ \frac{(n-1)}{v} & \text{otherwise} \end{cases} \quad (20)$$

Proof sketch. For $u = 0$ and $R_b < \frac{(2n-4)}{n^2}$ the DCT of BvN-SYS is lower than the DCT of RR-SYS and UPPER for $v < \frac{2}{n}$. Following [Observation 3](#), to prove the lemma, we only need to find the intersection point between the DCT of BvN-SYS, $1 + R_b v$ and the DCT of RR-SYS where the DIRECT schedule archives the minimum, i.e., the DCT is $\frac{n-1}{v}$. Solving for v gives the result and ends the proof sketch. To find the system DCTs as stated in [Theorem 8](#), we note that the worst-case matrix is when $v = \frac{\sqrt{1+4R_b(n-1)}-1}{2R_b}$ and the DCT is then $\Delta(\text{COMP-SYS}, \mathcal{M}) = 1 + \frac{\sqrt{1+4R_b(n-1)}-1}{2R_b} R_b = \frac{\sqrt{1+4R_b(n-1)}+1}{2}$.

To conclude, we see that our result and proof of [Theorem 8](#) show that RR-SYS and BvN-SYS have a system DCT about 22% higher than COMP-SYS for our chosen parameters and the family $M(v, u)$. However, this numerical result is only for our ad-hoc set of parameters. We may want to explore what happens to this ratio in other cases. First, let us denote the ratio between system DCT as ψ . This is the minimum between the ratio of the system DCT of RR-SYS to COMP-SYS and BvN-SYS to COMP-SYS, that is

$$\psi = \min\left(\frac{\Delta(\text{RR}, \mathcal{M})}{\Delta(\text{COMP-SYS}, \mathcal{M})}, \frac{\Delta(\text{BvN}, \mathcal{M})}{\Delta(\text{COMP-SYS}, \mathcal{M})}\right) - 1 \quad (21)$$

Where 1 was reduced such that $0 \leq \psi \leq 1$.

We make the following observation regarding our parameter set and the ratio of systems DCT.

Observation 5 (Limit of ψ). *Let $\mathcal{M}$ be the family of all $M(v, u)$ matrices. Assuming that the BvN reconfiguration time is bounded by $R_b < \frac{(2n-4)}{n^2}$. ψ is maximized when $R_b = \frac{1}{n}$. For this value of R_b when the number of nodes n increases*

$$\lim_{n \rightarrow \infty} \psi = \sqrt{5} - 2 \approx 0.236 \quad (22)$$

This can be deduced by finding the maximal value for ψ from [Equation 21](#), which achieved when $\Delta(\text{RR}, \mathcal{M}) = \Delta(\text{BvN}, \mathcal{M})$, and as a result, the BvN reconfiguration rate must be set to $R_b = \frac{1}{n}$. This observation, combined with our main result in [Theorem 8](#), shows that COMP-SYS maintains its advantage over the other systems for any system size.

7. Empirical Evaluation

In this section, we evaluate the performance of COMP-SYS and use BvN-SYS and RR-SYS as benchmarks.

Our COMP-SYS algorithm, PIVOT, takes a relatively global approach when scheduling traffic; it looks at combined sets of permutations and makes a decision that takes into account the uniformity of the combined traffic. In this section, we will also attempt to evaluate the positive effect of the global view taken by PIVOT. An COMP-SYS algorithm can schedule each permutation greedily and **independently** based on whether it will be served faster on RR-SYS or on BvN-SYS. Thus, in this section, for COMP-SYS, we will consider two algorithms, the novel PIVOT, which is already described in [Section 5](#), and GreedyThresholdNet (GTN), which we describe formally in [Appendix G](#). As we later explain, this local view means that GTN filters the permutations from the BvN decomposition based on a predetermined *threshold*, while this has

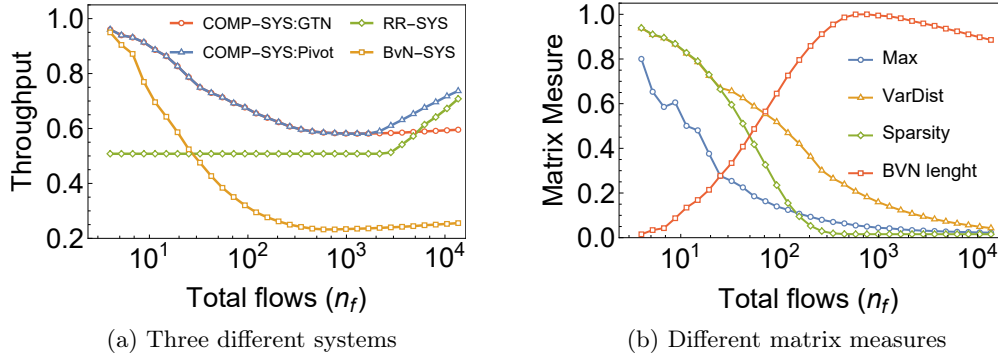


Figure 3: (a) The throughput of the three systems on matrices generated by the traffic model. Where COMP-SYS was tested with both PIVOT and GTN . (b) Four different measures of the demand matrices from (a).

the advantage of a shorter runtime, as we will see, it is gained by sacrificing certain benefits. This local, threshold-based, view is more in line with some related works, on topology engineering, such as TROD (using threshold-VLB) [39, 40], Cerberus [16] (routing each flow based on a threshold), and D3 [41] (routing permutations based on a threshold). These papers perform traffic routing decisions based on a threshold. However, the settings of each paper differ in multiple ways. An exact compression of COMP-SYS, Cerberus, TROD, and D3 is reserved for future work. We further note that PIVOT cannot be typically reduced to a pure threshold-based algorithm. We explain this with greater detail in [Appendix H](#). Throughout this section, we will use COMP-SYS:PIVOT and COMP-SYS:GTN to denote COMP-SYS using PIVOT or GTN, respectively. We begin by describing the traffic generation model and then present the results of our evaluation.

7.1. Traffic Generation

Our traffic generation model is based on the model used in Eclipse [18] and in [27] and denoted as $TM(t_l, n_f, c_l, n)$. A demand matrix is generated using the following four parameters. n_f : the total number of flows per node, t_l : the fraction of large flows per node, where $0 \leq t_l \leq 1$, and c_l : the load of large flow traffic, e.g., where if $c_l = 0.7$, then the large flows have 70% of all traffic in terms of bits. Lastly, n is the size of the square demand matrix. We define the number of large flows as $n_l = \lceil t_l n_f \rceil$, and the number of small flows as $n_s = n_f - \lceil t_l n_f \rceil$. The matrix is generated by adding n_f uniformly random derangement permutation matrices P_i . Each permutation is weighted by its type, whether small or large, plus additive white Gaussian noise to each permutation. The weight of a large flow permutation is, therefore, the following $\frac{t_l}{n_l} + \mathcal{N}(0, \sigma_l^2)$, where we set $\sigma_l = \lambda \frac{t_l}{n_l}$. We use a similar weight for the small flows where $\sigma_s = \lambda \frac{1-t_l}{n_s}$. Where λ is the deviation from the mean weight. We use $\lambda = 0.01$, meaning the standard deviation is about 1% of the average. This might generate a slightly smaller or larger matrix than a doubly stochastic matrix; therefore, we normalize its weight to one.

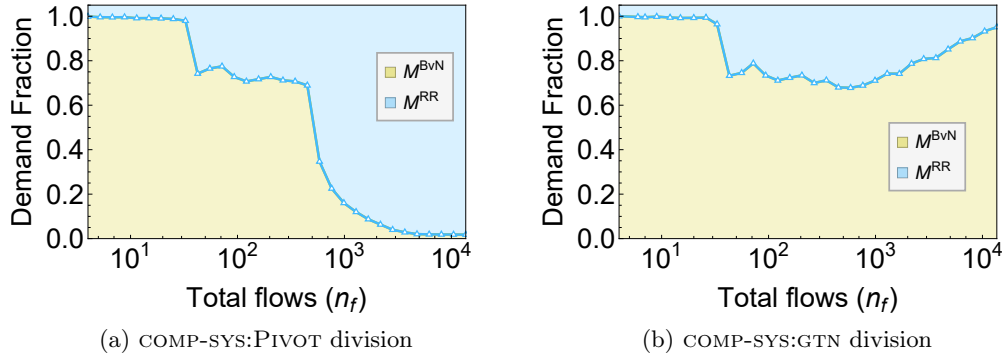


Figure 4: The division of the demand to M^{BvN} and M^{RR} , by (a) PIVOT and (b) GTN.

7.2. Evaluation

In this section, we discuss our empirical results. We will first see results on a chosen set of parameters and then show a sensitivity analysis. In the evaluation, we use the “birkhoff+” algorithm from the Julia package “Birkhoff decomposition” as described in [27]. Their BvN algorithm creates what we described as an ϵ schedule, i.e., a schedule that leaves a small amount of leftover traffic that is not served. We define this formally in Definition 12. We note that our evaluation uses the same ϵ value as used in the original paper, which is $\epsilon = 10^{-4}$.

Our main set of parameters is similar to previous work, such as in [27, 18]. Unless otherwise stated, We set the fraction of large flows t_l to 20%, and the load of large flows c_l to 70%.

In Figure 3 (a), we see the results of the three systems on traffic generated from our model. The y axis represents the throughput, and the x axis represents the number of flows generated per host. In this simulation, we ran up to $4n^2$ flows per host and repeat the simulation 30 times. We use $n = 64$, and for BvN-SYS we use a reconfiguration time of $R_b = 0.01s$.² For RR-SYS we set $\eta = 1$, that is $R_r = 0$. In short, the traffic is generated from the model with parameters $\mathcal{T} = \text{TM}(t_l = 0.2, n_f = x, c_l = 0.7, n = 64)$, where x is our changing parameter and $4 \leq x \leq 4n^2$. First and foremost, we see that COMP-SYS:PIVOT, (i.e., COMP-SYS using PIVOT) consistently achieve higher throughput than RR-SYS and BvN-SYS. That is, the ability to decompose and serve the matrix using two different types of schedules improves the throughput across the board. Between the two COMP-SYS algorithms, we see that PIVOT achieves better results than GTN for a dense traffic matrix. Essentially, when the matrix is dense, the BvN decomposition generates many permutations, and while GTN only makes decisions locally, based on single permutation matrices and a set threshold, PIVOT makes the decision based on sets of many permutations, allowing it to take advantage of more uniform traffic which appears when combining large numbers of matrices (in the context of our traffic

²We note that for the R_b we use a similar value as the related work [18, 27], and that using this value in the previous analysis for $M(v, u)$ would have only improved the result further for COMP-SYS.

model). We now use [Figure 3 \(b\)](#) to help us explain the results. This figure shows the same x axis, and on the y axis, we see several normalized measures of the demand matrix. The figure presents i) Sparsity: the fraction of empty (zero) cells in the matrix. ii) Max: the value of the maximal element in the matrix. iii) BVN length: The number of elements in the BVN decomposition divided by the length of the longest decomposition among all those tested. iv) VarDist: the Variation distance of the matrix from the uniform matrix. We observe using these measures that as we increase the number of flows, the generated matrices become more dense and uniform, and BVN needs a larger number of permutations. Looking at the [Figure 3 \(a\)](#), on the curves of both BVN-SYS and RR-SYS, we can observe behavior that is similar in essence to the expected behavior from [Figure 2 \(a\)](#). However, the crossing point between RR-SYS and BVN-SYS is very different here. The different traffic models can explain this difference in the results. In our current model, TM, traffic flows are generated uniformly at random. As a result, some flows may overlap and increase the irregularity, thus decreasing the rate at which the maximal value is reduced. However, as more flows are introduced to the system, the more similar it becomes to $M(n-1)$ from the previous section.

Recall that PIVOT and GTN divide the original demand matrix M into two sub-matrices M^{BVN} and M^{RR} , each of which handles a different share of the traffic. In [Figure 4 \(a\)](#) and (b), we see how this division is made on the demand matrices we generate. The y axis shows the relative fraction of traffic scheduled on BVN-SYS or to RR-SYS by PIVOT, while the x axis is the number of flows per host, as before. Naturally, as less load is scheduled to BVN-SYS, more is scheduled to RR-SYS. For PIVOT in [Figure 4 \(a\)](#) it is possible to divide the curve into three regions: a region where traffic is almost exclusively scheduled with BVN-SYS (until about 50 flows), a region where both systems are used significantly (until about 200 flows), and a region where traffic is scheduled chiefly with RR-SYS. We can see that the transition from one region to another is relatively abrupt. This can be explained in the following manner: in the first region, there are few *small* flows; these flows are still relatively large, and thus, they are scheduled along with the large flows to BVN-SYS. In the second region, the result is slightly more predictable. We see that the load scheduled to BVN-SYS roughly fits with c_l , the load of the large flows ($c_l = 0.7$) generated in the model. This implies that, in this region, the *small* flows cross a threshold where they are small enough to be scheduled on RR-SYS more efficiently than with BVN-SYS. In the third region, more and more traffic is sent to RR-SYS. This implies that the matrix becomes more uniform as more flows are represented in the demand matrix, as we also see in [Figure 3 \(b\)](#). Applying the same regional division of the plot to GTN in [Figure 4 \(b\)](#), we observe that the first two regions are quite similar, indicating that the algorithms behave similarly in this phase. However, unlike PIVOT, which around 200 flows begins allocating an increasing fraction of demand to M^{RR} , GTN shifts more demand toward M^{BVN} . This is because, although the traffic becomes more uniform, GTN still operates on singular permutations, rather than the global uniform structure. Comparing [Figure 3 \(a\)](#) [Figure 4 \(a\)](#), we might expect the DCT of COMP-SYS:PIVOT to be the same as BVN-SYS in the first region and the same

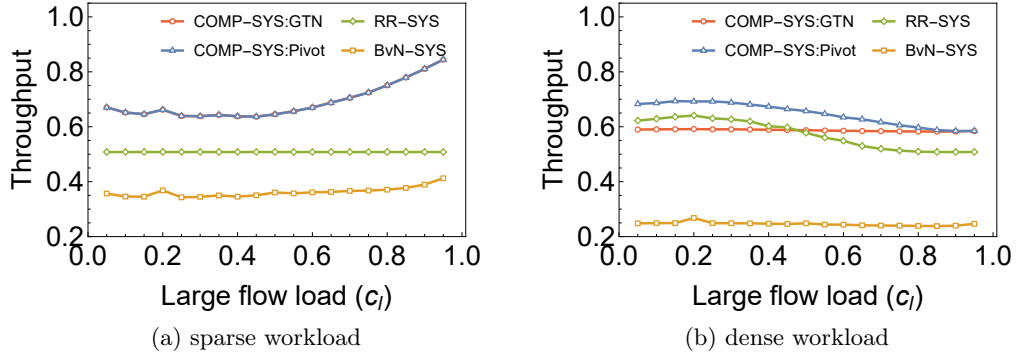


Figure 5: The throughput on a dense and a sparse workload, with a change in the large flow load c_l . note that for (a) COMP-SYS:Pivot, COMP-SYS:GTN overlap.

with RR-SYS in the third region. However, in the first region, there is a fraction of traffic, small in volume (related to the noise that we used in the traffic generation model), which COMP-SYS:Pivot can schedule with RR-SYS. In contrast, BvN-SYS must schedule any traffic with the cost of a switch reconfiguration, which incurs a delay of R_b per configuration regardless of the traffic volume sent. In the third region, the DCT of RR-SYS is becoming similar to that of COMP-SYS:Pivot. However, if the BvN decomposition can capture even a few large flows, scheduling them with BvN-SYS is still advantageous enough so that COMP-SYS:Pivot has a lower DCT than either BvN-SYS or RR-SYS, on the entire tested range.

To conclude, we can see that the worst case throughput for each system is $\frac{1}{\Delta(\text{COMP-SYS}, \mathcal{T})} \approx 0.58$ for COMP-SYS (both algorithms), $\frac{1}{\Delta(\text{RR-SYS}, \mathcal{T})} \approx 0.507$ for RR-SYS, and for BvN-SYS $\frac{1}{\Delta(\text{BvN-SYS}, \mathcal{T})} \approx 0.232$. The results for BvN-SYS are significantly different, which is expected as the total number of permutations rises, as we can also see in Figure 3 (b)³.

In the following part of this section, we study two other parameters of the model: the ratio (number) of large flows, t_l , and the load of the large flows c_l . We consider two important workload types. *sparse* workloads, where $n_f = 64$ and *dense* workloads where $n_f = 3000$. In Figure 5 (a) we test the following traffic model $\text{TM}(t_l = 0.2, n_f = 64, c_l = x, n = 64)$, that is, a relatively sparse workloads and in Figure 5 (b) and we test $\text{TM}(t_l = 0.2, n_f = 3000, c_l = x, n = 64)$, a dense workloads.

In Figure 5 (a) & (b), we see that COMP-SYS:Pivot achieves better throughput than either of the other systems, regardless of the value of c_l . In Figure 5 (a), the throughput of RR-SYS is constant at 0.507. This is because the matrices are sparse. In these kinds of matrices, there is a large gap between the maximal cell and average cell size, as seen in Figure 3 (b). We also see that both COMP-SYS and BvN-SYS improve with the larger c_l . For BvN-SYS, since in this setting the number of permutations generated by the model

³We take note of the interesting phenomenon in this context: when the n_f increases, at some point, the length of the schedule starts to decrease. As more and more permutations are added to generate the demand, the closer the matrix is to the uniform matrix $M(n-1)$ (as is also supported by the variation distance), which can be decomposed to only $n-1$ elements, less than the worst case.

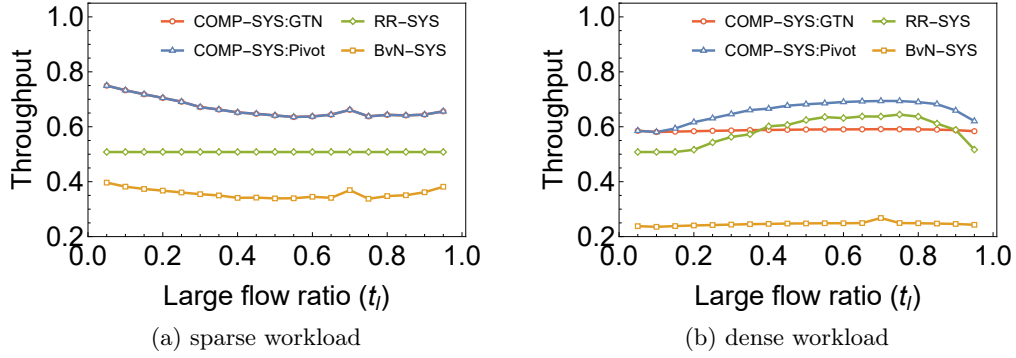


Figure 6: The throughput on a dense and a sparse workload, with a change in the large flow ratio t_l . Note that for (a) COMP-SYS:Pivot, COMP-SYS:GTN overlap.

is constant, we would expect no change in throughput; however, since the smaller permutations become so small when large flows become larger, they are discarded in the BvN decomposition since we perform an ϵ schedule. Any discarded permutation will improve the completion time, but as we can observe in the figure, this effect is limited. For COMP-SYS COMP-SYS:PIVOT we see a larger improvement than for BvN-SYS. Since the number of large permutations is static, the larger the load handled on these few permutations, the more effective COMP-SYS becomes, as this system more effectively amortizes the reconfiguration time for BvN-SYS. In contrast, unlike BvN-SYS, it can handle the fraction of load represented in the small flows more effectively via RR-SYS. Between the two COMP-SYS algorithms, PIVOT and GTN, again, the algorithms achieve identical results for sparse traffic. For the dense workloads setting in Figure 5 (b), we see that BvN-SYS is static, as the change in the load c_l doesn't affect the number of permutations. When the load c_l is small, these few matchings don't affect the shape of the matrix. Since the small flows are a majority in terms of numbers and load share, they form a more uniform matrix, which both RR-SYS and COMP-SYS can take advantage of in the same manner, hence their similar curves. For dense traffic, COMP-SYS:PIVOT achieves better results than COMP-SYS:GTN for traffic with a lower c_l load. As this load increases, both algorithms' performance becomes similar. This is likely due to COMP-SYS:GTN making less efficient use of the RR-SYS schedule, which is more prominent when the small flows represent a larger traffic load. Since COMP-SYS:GTN makes less effective use of RR-SYS, it is ineffective for traffic with a large fraction of small flows for about half the tested range.

In Figure 6 (a) & (b), we test what happens when we change the ratio of the number of large flows. In Figure 6 (a) we test the following traffic model $TM(t_l = x, n_f = 64, c_l = 0.7, n = 64)$, sparse workloads, and in Figure 6 (b) we test the following traffic model $TM(t_l = x, n_f = 3000, c_l = 0.7, n = 64)$, dense workloads. For Figure 6 (a), we see, as before in Figure 5 (a), that for the sparse workload, the RR-SYS result is static, and, COMP-SYS:PIVOT and COMP-SYS:GTN, achieve identical results for sparse traffic. Both COMP-SYS and BvN-SYS show a similar change in the shape of their curves. This change results from the fact that at both extremes of this traffic model, where the ratio of large flows tends to one or zero, the matrix

becomes almost like a matrix with a single type of traffic and, therefore, the BVN decomposition generates fewer permutations. However, throughout the tested range, COMP-SYS is able to serve a set of flows more effectively than BVN-SYS, achieving a marked advantage with throughput. In [Figure 6 \(b\)](#), we see a different effect. In the extremes of the tested range of t_l , there are *few* of either large or small flows, which means that the matrix is more uniform and, therefore, it benefits the throughput of RR-SYS. Interestingly, when the large flow ratio is small (0.05 or 0.1), the throughput of COMP-SYS:PIVOT and COMP-SYS:GTN are identical. This is likely since there are a few large flows which are all scheduled with the BVN-SYS aspect of COMP-SYS in both cases. Otherwise, COMP-SYS:PIVOT, with its more global view of traffic, is able to schedule some large flow traffic to RR-SYS, when this improves the uniformity of M^{RR} . For a more in-depth discussion of the superior performance of PIVOT over GTN, we refer to [Appendix H](#).

To conclude this section, COMP-SYS:GTN is unable to fully take advantage of uniform traffic, leading it to lose to RR-SYS in some cases, or being identical in others. Thus COMP-SYS:PIVOT is strictly better than COMP-SYS:GTN, in terms of throughput. COMP-SYS:PIVOT is also superior to RR-SYS, BVN-SYS, while changing the load and ratio parameter of the large flows. We see that for different workloads, different aspects of the COMP-SYS system become dominant, contributing to the change in throughput. Both RR-SYS and BVN-SYS showcase distinct strengths and weaknesses across various workloads. Leveraging their respective advantages, COMP-SYS:PIVOT can use them synergistically, resulting in better outcomes.

8. Bridging Theory and Practice

The primary contribution of this paper is its theoretical framework, for which we deliberately abstracted away certain practical concerns to yield clear and tractable results. In this section, we examine the impact of relaxing our core assumptions and assess the extent to which COMP-SYS aligns with current datacenter technologies and operational realities.

8.1. Effect of Model Assumptions on Results

Here, we will discuss our assumptions (as mentioned in [Section 2](#)) and their effect on our results for the systems RR-SYS, BVN-SYS, and COMP-SYS, if they are relaxed.

Idealized system: Our model abstracts away synchronization issues, packet loss, retransmissions, link failures, propagation delay, routing algorithm overheads, and physical layer concerns. While these factors are essential in practice, they do not affect our main takeaway regarding the throughput advantage of COMP-SYS over BVN-SYS and RR-SYS. Instead, our findings should be interpreted as a lower bound, which could be *approached* assuming a good solution to these practical issues.

Global reconfiguration delay: We assume that all circuits are unavailable in each reconfiguration period. This assumption reflects the behavior of some real-world MEMS-based OCS systems and was therefore chosen. However, it may change with future technologies such as wavelength-selective switching [\[42\]](#). A more

general model could assume link-by-link reconfiguration, where only affected links are briefly unavailable. Such a model would likely yield better throughput bounds by reducing the impact of reconfiguration delays.

Single matching, single hop: We assume that traffic in each matching can only traverse a single hop and that multi-hop forwarding is only through multiple, distinct matchings. Relaxing this assumption for BVN-SYS will have no effect, as traffic is always sent using a single hop. RR-SYS, using our UPPER scheduler as discussed in [Section 4](#), achieves the lower bound DCT for both the permutation and uniform matrices, thus all traffic will be sent continuously without idle time (i.e., full link utilization), and both DCT and throughput cannot be improved further by using multi-hop traffic. For other cases, the lower throughput or DCT bound for RR-SYS and the scheduler to achieve them are not known; thus, the answer depends on whether this schedule sends traffic continuously. If it follows the same trend as the known cases, throughput cannot be improved further using multi-hop traffic. But this remains an open question. Consequently, the effect on COMP-SYS would depend on the impact on RR-SYS. We offer an observation on an important metric not explored in this work: relaxing the single-hop assumption could improve the average flow completion times (FCT) in RR-SYS. Specifically, small flows may be delivered earlier if a multi-hop path exists at the time of the flow’s arrival, similar to the approach in Opera [\[22\]](#). For instance, a small flow could complete in the first configuration of the first round in an RR-SYS schedule, which is not possible under the one-hop constraint. Relaxing this assumption and optimizing our algorithms (or using novel ones) for better average FCT remains an interesting direction for future work.

Traffic matrix is of the family $M(v, u)$. We introduced the $M(v, u)$ matrix family as a structured case for mathematical analysis. While seemingly narrow, we claim that this family is relatively broad and can be used to capture fundamental traffic patterns. First, to make a simple observation: since all permutation matrices $M(1)$ are contained within the $M(v, u)$ family, it follows that any doubly stochastic matrix can be decomposed into a set of $M(v, u)$ matrices, and vice versa, any such matrix can be constructed from them. Importantly, $M(1)$ and $M(n - 1)$ which are at the two ends of the $M(v)$ spectrum, correspond to bursty/high-volume and all-to-all traffic patterns, respectively. The $M(v, u)$ family, which is a weighted combination of $M(v)$ and $M(n - 1)$, thus stands as a simplified representation of these two fundamental traffic patterns, which are common in many prior works, including [\[39, 12, 40, 16\]](#), and many others. As noted in [Section 6](#), $M(v, u)$ also includes each system’s specific worst and best-case theoretical scenarios. Thus it may seem that to take advantage of our systems effectively one would need to decompose a general doubly stochastic matrix into a set of $M(v, u)$ matrices⁴, which may be a complex problem, but this is not strictly necessary. Our empirical evaluation, in [Section 7.2](#), provided a practical approach using the PIVOT algorithm. We decompose a *general* doubly stochastic matrix into two components. The first approaches

⁴A formal characterization of the properties that make this decomposition optimal for our systems is beyond the scope of this work.

all-to-all traffic ($M(n-1)$) and the second represents high-volume/bursty traffic ($M(v)$ with $v < n-1$).

In summary, relaxing our core assumptions should not notably affect our main takeaway regarding the throughput advantage of COMP-SYS for systems based on RR-SYS, BVN-SYS, or COMP-SYS. However, future work may improve COMP-SYS on other metrics using more advanced models and algorithms. Naturally, we cannot comment on the effects for other system types.

8.2. Practicality and Scalability

This section will briefly discuss how our results can be translated into practical systems. We described our network as being based on a **leaf-spine** architecture, with a *single* spine OCS, with an n *radix*, capable of implementing *both* an RR-SYS and a BVN type schedules. Let us examine the scalability of this model.

As we mentioned, there is a body of literature that has shown an equivalence of results between a single n radix switch and a k -switch of the same radix [9, 35, 12, 36, 43]. We emphasize that this equivalence requires that all spine switches are identical in function, interconnection, and radix. However, the use of the single switch allowed us to simplify the analysis without loss of generality. Generally, the throughput of the network of k switches would be k times faster; however, since we deal with a scaled-down saturated traffic matrix, the traffic load would increase k fold, preserving our result.

Our approach to model a dual-role spine switch, with the ability to support *both* scheduling types is rooted in prior work [12, 32]. Although every switch has a fixed, hardware-related reconfiguration delay, the gap between BVN type and round-robin type schedules arises because BVN requires online computation of an ad-hoc schedule of matchings, whereas RR-SYS uses a predefined, precomputed schedule, hence its faster reconfiguration. To ensure the validity of our analysis for a multi-switch network (when implementing COMP-SYS), we assume that the role of each switch is dynamic, alternating between BVN-SYS and RR-SYS schedules according to the demand. The implementation of a heterogeneous topology, with dedicated and different types of switches or switches of different radix may be beneficial, for example, since the performance of RR-SYS type systems can be further enhanced by dedicated hardware implementations [44]. However, we leave this analysis as future work.

A main scalability challenge of the *leaf-spine architecture* is that each switch (either at the spine or leaf layer) must support a radix of n . For the spine layer, high-radix optical circuit switches (OCS) are under active development: current MEMS-based OCSs already reach 576×576 ports [45], and recent RR-SYS-like prototype could exceed 1k ports [44], with further growth expected. Alternatively, three-tier topologies that insert an aggregation layer (to pods) between spine and leaf layers can reduce the per-switch radix requirement for both layers [40, 39, 10, 36], and a similar, multi-tier approach has been proposed for RR-SYS-like architectures [46]. We believe that in this scaled multi-tier architecture, our analysis would stretch to cover the top spine layer, with the lower aggregation layer standing in for our leaf layer. We leave this for future work.

The scalability of PIVOT mirrors that of other systems using BvN decomposition, e.g., Eclipse [18], inheriting their computational overhead for matching and reliance on global demand information. Specifically, our solution requires a centralized SDN controller and real-time measurements of ToR-level traffic demand, which are all feasible for certain scales and accuracy levels [10, 47].

9. Related work

Traditional DCN topologies are static and predominantly based on different flavors of Clos or fat tree topologies [3, 4, 5], with some more recent proposals using expander graphs [6, 48]. More recently, Dynamic OCSs allowed the advent of reconfigurable DCNs. (RDCNs) [9, 11].

The two basic approaches to topology engineering in RDCNs are *demand-aware* and *demand-oblivious* topology designs [13]. In a nutshell, demand-aware topologies are dynamic networks that adjust the topology to the actual demand, optimizing the topology to particular traffic patterns by rewiring connections and changing the topology. On the contrary, demand-oblivious topologies [11] present the same topology to any traffic patterns, oblivious to the actual demand. Note, however, that demand-oblivious systems can be dynamic (e.g., RotorNet [14]) or static (e.g., Clos [3]).

Demand-aware networks and BvN type Systems. Typical examples of demand-aware topologies are Eclipse [18], projecTor [17], Morida [23], and others [49, 20, 21]. The topology in these systems can change dynamically. This topology is a union of matching generated from a schedule of matchings where all or most traffic flows are served in a direct connection, i.e., single hop. To construct the topology schedule, some works treat this problem as a max weighted matching problem on bipartite graphs, such as in Helios [35], or an online stable matching problem, such as in ProjecTor. In Eclipse [18] and FireFly [21], the authors use Birkhoff von Neumann (BvN) decomposition to create a schedule of matchings, which we denote in this paper as BvN-SYS type systems. While BvN saw a wide array of applications in several fields in the context of dynamic networks, it was often used to generate a schedule of matchings tailored to the demand of the network [50, 21, 18, 20, 26, 51]. In the context of dynamic networks, to be efficient, a schedule needs not only to cover all the traffic but also to use a minimal number of switch reconfigurations, which is a known NP-hard problem [52, 53]. In a recent paper, [27], Valls et al. attempt to optimize BvN and construct a schedule in a less naive way than previous works, allowing for fewer configurations (fewer matchings) and faster run time. Indeed, the run time of the generation process for a novel schedule itself could have a non-negligible effect on the reconfiguration time, which was also improved in [27], over simpler methods used in Eclipse [18] and Solstice [20]. Reconfiguring the network may result in lost packets, Lv et-al [54], introduced a hitless topology-and-path engineering scheme that ensures zero packet loss during reconfiguration by guaranteeing that each flow always has a valid optical path. More recently, reconfigurable DCN architectures have been tested for ML training workloads (e.g., large-language-model training), and have demonstrated performance

gains over static networks [55, 43].

The reconfiguration time is an important factor limiting demand-aware reconfigurable networks, which is limited by both hardware and software. The hardware (i.e., the OCS) must deal with moving physical mirrors, vibrations causing interference, etc. [17]. The 3D MEMES switch [38]) allows, in principle, arbitrary reconfigurations of the topology in periods on the scale of *micro-seconds*. From a software perspective, algorithms need to generate a topology schedule, collect demand [9], etc., computations that might be significant compared to the physical reconfiguration time, as we have mentioned regarding BvN. Another approach to OCS reconfiguration, is to make *infrequent* and slow reconfigurations. In [10], Google has reported that topology changes may be needed once every few hours or days for their datacenters. Other works perform robust topology optimization, that is, they attempt to minimize worst-case performance (typically throughput) with infrequent reconfigurations [39, 36, 56] have had a similar conclusion using datacenter traces.

Demand-Oblivious Networks and Round-Robin Type Systems. Other systems, which we termed in this paper as RR-SYS (e.g., RotorNet), can achieve *nono-seconds* scale reconfiguration times. This was recently demonstrated by a prototype in [44]. They achieve this mainly by using a predefined topology schedule of matchings and going through it periodically in a round-robin manner which allows the system to save on computation, and perform hardware-level optimizations that take advantage of this predictability. In this paper, we assumed that the union of the matchings in one period of the topology schedule is a complete graph; however, this is not always the case, such as in Mars [15], where the graph could be of a lower degree. In Sirius [12], RotorNet [14], and Opera [22], the schedule generates full mesh connectivity over time. Therefore, packets may be sent directly to any node in one hop. However, this may not be very efficient in every case, for example, when the matrix is skewed. In a skewed matrix, load-balancing methods, such as Valiant load balancing (VLB), are employed. For example, in Sirius, all packets are sent at two hops, while for RotorNet and Opera, only packets that cannot be sent at a single hop are sent at two hops. In other works, such as in [57, 58], the authors propose a ‘traffic-aware’ load balancing scheme, which extends VLB, for leaf-spine DCN. This scheme can reduce retransmissions and flow completion times. In [46], the authors discuss a more generalized version of round-robin networks. In this work, rather than each node participating in a round-robin schedule between all nodes, each node participates in a partial schedule, where only a part of the nodes are connected. This allows the system to achieve better latency at a cost of lower throughput.

Composite Systems. This paper considered the composite system a more flexible reconfigurable network design, which we denoted as COMP-SYS. These systems combine the topology engineering approaches of both round-robin-like systems and BvN demand-aware-like systems. A close example of a COMP-SYS type system is Cerberus [16]. In Cerberus, the authors combine several network designs, rotor, demand-aware (not based

on BvN), and, to a lesser extent, a static expander into a single unified system. The paper shows that throughput can be increased when traffic is routed towards the most appropriate sub-network. However, Cerberus did not offer an accurate, in-depth analysis of the bounds on the performance of its composite system in terms of demand completion time.

Performance Metrics. As we discussed in this section, there exists a large body of work on the design of communication networks, different metrics on which to compare the performance of different network designs [59, 33, 37], e.g., bisection bandwidth and other cut-based metrics. However, these give little information regarding the worst-case throughput performance [33]. In their work, Jyothi et al. [33] define throughput, in short, as the maximal scale-down factor for a demand matrix (traffic matrix), for which the matrix is feasible, a definition also used in some more recent works [16, 15, 37]. The authors note that finding the worst-case matrix for a given network is difficult. Therefore, they present ways to find a near-worst-case traffic matrix for several known systems. The more recent paper by Namyar et al. [37] further explores the relation between bisection bandwidth and throughput. The authors also note that the throughput of reconfigurable demand-aware systems is an open problem. We note that our paper is supposed to be a step in this direction.

10. Conclusions

This paper proposes a simple model to compare and study the tradeoff between demand-aware and demand-oblivious topologies and traffic schedules. Our model can be seen as an extension of the well-known BvN-decomposition problem and an attempt to integrate Topology and Traffic engineering in a simple model. We rigorously define and analyze the throughput bounds of BvN-SYS and RR-SYS for double stochastic matrices. We give bounds on the throughput of COMP-SYS for the $M(v, u)$ matrix family, which, importantly, captures both edge case workloads of BvN-SYS and RR-SYS, and prove that COMP-SYS has higher throughput than either of the two systems. Our empirical analysis further shows COMP-SYS will achieve higher throughput for a wide range of model parameters and traffic matrices. Nevertheless, many open questions remain. First and foremost, what is the worst-case (double stochastic) demand matrix for COMP-SYS? Another interesting direction is to extend our analysis to a system of multiple spine switches, which may be of a smaller (than n) radix. We leave these challenges to future work, hoping that the framework and algorithms presented here will serve as a foundation for further progress.

Acknowledgement: This project received funding from the Israel Science Foundation (ISF) grant no. 2497/23.

References

- [1] E. Timurdogan, “Unleashing ai data center growth through optics,” <https://www.laserfocusworld.com/optics/article/14300952/unleashing-ai-data-center-growth-through-optics>.

- [2] A. Bjorlin, "Infrastructure for large scale ai: "empowering open"," in *Open Compute Project*, San Jose, CA, October 2022.
- [3] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," *ACM SIGCOMM computer communication review*, vol. 38, no. 4, pp. 63–74, 2008.
- [4] V. Liu, D. Halperin, A. Krishnamurthy, and T. Anderson, "F10: A fault-tolerant engineered network," in *10th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 13)*, 2013, pp. 399–412.
- [5] A. Singh, J. Ong, A. Agarwal, G. Anderson, A. Armistead, R. Bannon, S. Boving, G. Desai, B. Felderman, P. Germano *et al.*, "Jupiter rising: A decade of clos topologies and centralized control in google's datacenter network," *ACM SIGCOMM computer communication review*, vol. 45, no. 4, pp. 183–197, 2015.
- [6] S. Kassing, A. Valadarsky, G. Shahaf, M. Schapira, and A. Singla, "Beyond fat-trees without antennae, mirrors, and disco-balls," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, 2017, pp. 281–294.
- [7] T. Benson, A. Anand, A. Akella, and M. Zhang, "Microte: Fine grained traffic engineering for data centers," in *Proceedings of the seventh conference on emerging networking experiments and technologies*, 2011, pp. 1–12.
- [8] A. Singla, C.-Y. Hong, L. Popa, and P. B. Godfrey, "Jellyfish: Networking data centers randomly," in *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, 2012, pp. 225–238.
- [9] G. Wang, D. G. Andersen, M. Kaminsky, M. Kozuch, T. Ng, K. Papagiannaki, M. Glick, and L. Mummert, "Your data center is a router: The case for reconfigurable optical circuit switched paths," *Proc. ACM Hotnets VIII*, 2009.
- [10] L. Poutievski, O. Mashayekhi, J. Ong, A. Singh, M. Tariq, R. Wang, J. Zhang, V. Beauregard, P. Conner, S. Gribble *et al.*, "Jupiter evolving: Transforming google's datacenter network via optical circuit switches and software-defined networking," in *Proceedings of the ACM SIGCOMM 2022 Conference*, 2022, pp. 66–85.
- [11] M. N. Hall, K.-T. Foerster, S. Schmid, and R. Durairajan, "A survey of reconfigurable optical networks," *Optical Switching and Networking*, vol. 41, p. 100621, 2021.

- [12] H. Ballani, P. Costa, R. Behrendt, D. Cletheroe, I. Haller, K. Jozwik, F. Karinou, S. Lange, K. Shi, B. Thomsen *et al.*, “Sirius: A flat datacenter network with nanosecond optical switching,” in *Proceedings of the ACM SIGCOMM 2020 Conference*, 2020, pp. 782–797.
- [13] C. Avin and S. Schmid, “Toward demand-aware networking: A theory for self-adjusting networks,” *ACM SIGCOMM Computer Communication Review*, vol. 48, no. 5, pp. 31–40, 2019.
- [14] W. M. Mellette, R. McGuinness, A. Roy, A. Forencich, G. Papen, A. C. Snoeren, and G. Porter, “Rotornet: A scalable, low-complexity, optical datacenter network,” in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, 2017, pp. 267–280.
- [15] V. Addanki, C. Avin, and S. Schmid, “Mars: Near-optimal throughput with shallow buffers in reconfigurable datacenter networks,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 7, no. 1, pp. 1–43, 2023.
- [16] C. Griner, J. Zerwas, A. Blenk, M. Ghobadi, S. Schmid, and C. Avin, “Cerberus: The power of choices in datacenter topology design—a throughput perspective,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 5, no. 3, pp. 1–33, 2021.
- [17] M. Ghobadi, R. Mahajan, A. Phanishayee, N. Devanur, J. Kulkarni, G. Ranade, P.-A. Blanche, H. Rastegarfar, M. Glick, and D. Kilper, “Projector: Agile reconfigurable data center interconnect,” in *Proceedings of the 2016 ACM SIGCOMM Conference*, 2016, pp. 216–229.
- [18] S. Bojja Venkatakrishnan, M. Alizadeh, and P. Viswanath, “Costly circuits, submodular schedules and approximate carathéodory theorems,” in *Proceedings of the 2016 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Science*, 2016, pp. 75–88.
- [19] J. Zerwas, C. Györgyi, A. Blenk, S. Schmid, and C. Avin, “Duo: A high-throughput reconfigurable datacenter network using local routing and control,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 7, no. 1, pp. 1–25, 2023.
- [20] H. Liu, M. K. Mukerjee, C. Li, N. Feltman, G. Papen, S. Savage, S. Seshan, G. M. Voelker, D. G. Andersen, M. Kaminsky *et al.*, “Scheduling techniques for hybrid circuit/packet networks,” in *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies*, 2015, pp. 1–13.
- [21] N. Hamedazimi, Z. Qazi, H. Gupta, V. Sekar, S. R. Das, J. P. Longtin, H. Shah, and A. Tanwer, “Firefly: A reconfigurable wireless data center fabric using free-space optics,” in *Proceedings of the 2014 ACM conference on SIGCOMM*, 2014, pp. 319–330.

- [22] W. M. Mellette, R. Das, Y. Guo, R. McGuinness, A. C. Snoeren, and G. Porter, “Expanding across time to deliver bandwidth efficiency and low latency,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020, pp. 1–18.
- [23] N. Farrington, A. Forencich, G. Porter, P.-C. Sun, J. E. Ford, Y. Fainman, G. C. Papen, and A. Vahdat, “A multiport microsecond optical circuit switch for data center networking,” *IEEE Photonics Technology Letters*, vol. 25, no. 16, pp. 1589–1592, 2013.
- [24] M. Frank and P. Wolfe, “An algorithm for quadratic programming,” *Naval research logistics quarterly*, vol. 3, no. 1-2, pp. 95–110, 1956.
- [25] G. Birkhoff, “Tres observaciones sobre el algebra lineal,” *Univ. Nac. Tucuman, Ser. A*, vol. 5, pp. 147–154, 1946.
- [26] A. Livshits and S. Vargaftik, “Lumos: A fast and efficient optical circuit switch scheduling technique,” *IEEE Communications Letters*, vol. 22, no. 10, pp. 2028–2031, 2018.
- [27] V. Valls, G. Iosifidis, and L. Tassiulas, “Birkhoff’s decomposition revisited: Sparse scheduling for high-speed circuit switches,” *IEEE/ACM Transactions on Networking*, vol. 29, no. 6, pp. 2399–2412, 2021.
- [28] V. Addanki, C. Avin, and S. Schmid, “Mars: Near-optimal throughput with shallow buffers in reconfigurable datacenter networks,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 7, no. 1, pp. 1–43, 2023.
- [29] L. G. Valiant and G. J. Brebner, “Universal schemes for parallel communication,” in *Proceedings of the thirteenth annual ACM symposium on Theory of computing*, 1981, pp. 263–277.
- [30] Y. Azar, E. Cohen, A. Fiat, H. Kaplan, and H. Racke, “Optimal oblivious routing in polynomial time,” in *Proceedings of the thirty-fifth annual ACM symposium on Theory of computing*, 2003, pp. 383–388.
- [31] J. P. Lang, V. Sharma, and E. A. Varvarigos, “An analysis of oblivious and adaptive routing in optical networks with wavelength translation,” *IEEE/ACM Transactions on networking*, vol. 9, no. 4, pp. 503–517, 2001.
- [32] C. Liang, X. Song, J. Cheng, M. Wang, Y. Liu, Z. Liu, S. Zhao, and Y. Cui, “Negotiator: Towards a simple yet effective on-demand reconfigurable datacenter network,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 415–432.
- [33] S. A. Jyothi, A. Singla, P. B. Godfrey, and A. Kolla, “Measuring and understanding throughput of network topologies,” in *SC’16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2016, pp. 761–772.

- [34] C. Griner, “Integrated-topology-and-traffic-engineering-for-reconfigurable-datacenter-networks,” <https://github.com/KhenGR/Integrated-Topology-and-Traffic-Engineering-for-Reconfigurable-Datacenter-Networks.git>, 2024.
- [35] N. Farrington, G. Porter, S. Radhakrishnan, H. H. Bazzaz, V. Subramanya, Y. Fainman, G. Papen, and A. Vahdat, “Helios: a hybrid electrical/optical switch architecture for modular data centers,” in *Proceedings of the ACM SIGCOMM 2010 Conference*, 2010, pp. 339–350.
- [36] S. Zhao, P. Cao, and X. Wang, “Understanding the performance guarantee of physical topology design for optical circuit switched data centers,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 5, no. 3, pp. 1–24, 2021.
- [37] P. Namyar, S. Supittayapornpong, M. Zhang, M. Yu, and R. Govindan, “A throughput-centric view of the performance of datacenter topologies,” in *Proc. ACM SIGCOMM Conference*, 2021, p. 349–369.
- [38] R. Ryf, J. Kim, J. Hickey, A. Gnauck, D. Carr, F. Pardo, C. Bolle, R. Frahm, N. Basavanthally, C. Yoh *et al.*, “1296-port mems transparent optical crossconnect with 2.07 petabit/s switch capacity,” in *OFC 2001. Optical Fiber Communication Conference and Exhibit. Technical Digest Postconference Edition (IEEE Cat. 01CH37171)*, vol. 4. IEEE, 2001, pp. PD28–PD28.
- [39] P. Cao, S. Zhao, M. Y. The, Y. Liu, and X. Wang, “Trod: Evolving from electrical data center to optical data center,” in *2021 IEEE 29th International Conference on Network Protocols (ICNP)*. IEEE, 2021, pp. 1–11.
- [40] P. Cao, S. Zhao, D. Zhang, Z. Liu, M. Xu, M. Y. Teh, Y. Liu, X. Wang, and C. Zhou, “Threshold-based routing-topology co-design for optical data center,” *IEEE/ACM Transactions on Networking*, vol. 31, no. 6, pp. 2870–2885, 2023.
- [41] J. Zerwas, C. Griner, S. Schmid, and C. Avin, “D3: An adaptive reconfigurable datacenter network,” *arXiv preprint arXiv:2406.13380*, 2024.
- [42] A. Shakeri, M. Garrich, A. Bravalheri, D. Careglio, J. Solé-Pareta, and A. Fumagalli, “Traffic allocation strategies in wss-based dynamic optical networks,” *Journal of Optical Communications and Networking*, vol. 9, no. 4, pp. B112–B123, 2017.
- [43] X. Han, S. Zhao, Y. Lv, P. Cao, W. Jiang, S. Lin, and X. Wang, “Lumoscore: Highly scalable llm clusters with optical interconnect,” *arXiv e-prints*, pp. arXiv–2411, 2024.
- [44] W. M. Mellette, A. Forencich, R. Athapathu, A. C. Snoeren, G. Papen, and G. Porter, “Realizing rotnet: Toward practical microsecond scale optical networking,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 392–414.

- [45] Polatis Ltd., “Huber+suhrer creates ground-breaking polatis optical switch platform with industry-leading capacity,” https://www.polatis.com/press-releases/HUBER_SUHRER_creates-ground-breaking_POLATIS_optical_switch_platform_with_industry-leading_capacity.asp, 2025, accessed: 2025-07-18.
- [46] D. Amir, N. Saran, T. Wilson, R. Kleinberg, V. Shrivastav, and H. Weatherspoon, “Shale: A practical, scalable oblivious reconfigurable network,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 449–464.
- [47] C. Avin and S. Schmid, “Revolutionizing datacenter networks via reconfigurable topologies,” *Communications of the ACM*, vol. 68, no. 6, pp. 44–53, 2025.
- [48] M. Zhang, R. N. Mysore, S. Supittayapornpong, and R. Govindan, “Understanding lifecycle management complexity of datacenter topologies,” in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019, pp. 235–254.
- [49] X. Zhou, Z. Zhang, Y. Zhu, Y. Li, S. Kumar, A. Vahdat, B. Y. Zhao, and H. Zheng, “Mirror mirror on the ceiling: Flexible wireless links for data centers,” *ACM SIGCOMM Comput. Commun. Rev. (CCR)*, vol. 42, no. 4, pp. 443–454, 2012.
- [50] C.-S. Chang, D.-S. Lee, and Y.-S. Jou, “Load balanced birkhoff–von neumann switches, part i: One-stage buffering,” *Computer Communications*, vol. 25, no. 6, pp. 611–622, 2002.
- [51] G. Porter, R. Strong, N. Farrington, A. Forencich, P. Chen-Sun, T. Rosing, Y. Fainman, G. Papen, and A. Vahdat, “Integrating microsecond circuit switching into the data center,” *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 447–458, 2013.
- [52] F. Dufossé and B. Uçar, “Notes on birkhoff–von neumann decomposition of doubly stochastic matrices,” *Linear Algebra and its Applications*, vol. 497, pp. 108–115, 2016.
- [53] J. Kulkarni, E. Lee, and M. Singh, “Minimum birkhoff-von neumann decomposition,” in *Integer Programming and Combinatorial Optimization: 19th International Conference, IPCO 2017, Waterloo, ON, Canada, June 26-28, 2017, Proceedings 19*. Springer, 2017, pp. 343–354.
- [54] Q. Lv, Y. Zhang, S. Zhang, R. Li, K. Meng, B. Zhang, F. Huang, X. Chen, and Z. Zhu, “On the tpe design to efficiently accelerate hitless reconfiguration of ocs-based dcns,” *IEEE Journal on Selected Areas in Communications*, 2025.
- [55] W. Wang, M. Ghobadi, K. Shakeri, Y. Zhang, and N. Hasani, “Rail-only: A low-cost high-performance network for training llms with trillion parameters,” in *2024 IEEE Symposium on High-Performance Interconnects (HOTI)*. IEEE, 2024, pp. 1–10.

- [56] M. Y. Teh, S. Zhao, P. Cao, and K. Bergman, “Enabling quasi-static reconfigurable networks with robust topology engineering,” *IEEE/ACM Transactions on Networking*, vol. 31, no. 3, pp. 1056–1070, 2022.
- [57] J. Hu, C. Zeng, Z. Wang, J. Zhang, K. Guo, H. Xu, J. Huang, and K. Chen, “Load balancing with multi-level signals for lossless datacenter networks,” *IEEE/ACM Transactions on Networking*, vol. 32, no. 3, pp. 2736–2748, 2024.
- [58] J. Hu, Y. He, W. Luo, J. Huang, and J. Wang, “Enhancing load balancing with in-network recirculation to prevent packet reordering in lossless data centers,” *IEEE/ACM Transactions on Networking*, vol. 32, no. 5, pp. 4114–4127, 2024.
- [59] X. Yuan, S. Mahapatra, M. Lang, and S. Pakin, “Lfti: A new performance metric for assessing interconnect designs for extreme-scale hpc systems,” in *2014 IEEE 28th International Parallel and Distributed Processing Symposium*. IEEE, 2014, pp. 273–282.
- [60] M. Marcus and R. Ree, “Diagonals of doubly stochastic matrices,” *The Quarterly Journal of Mathematics*, vol. 10, no. 1, pp. 296–302, 1959.

A. Preliminaries

We recall some relevant, important definitions.

Doubly stochastic matrix. A square matrix M of non-negative numbers is a *doubly stochastic matrix* (aka bistochastic) if the sum of the cells $M[i, j]$ in each row and column is equal to one, i.e., $\sum_i M[i, j] = \sum_j M[i, j] = 1$.

Saturated matrix and scaled matrix. A λ *scaled* doubly stochastic matrix is a matrix where the sum of each row is λ . In network traffic, a demand matrix M is called *saturated* if it is λ scaled and $\lambda = r$, where r is the transmission rate of all nodes, i.e., the sum of each row and column, will be r .

Permutation Matrix. A *permutation matrix* P is a double stochastic matrix where each row and column has exactly one cell with a value of 1 and zero otherwise. We denote the set of all derangement permutation matrices (without fixed points) as P_π .

The Weight of a Matrix and a Matrix Set. For a demand matrix M we denote its *weight*, $\|M\|_\Sigma$, as the sum of all its elements, $\|M\|_\Sigma = \sum_{ij} M[i, j]$. For a set $\overline{M} = \{m_1, \dots, m_v\}$ of such matrices we define $\|\overline{M}\|_\Sigma = \sum_{i=1}^v \|m_i\|_\Sigma$.

Birkhoff-von Neumann Decomposition. The Birkhoff-von Neumann theorem [25] (or BvN), states that given a doubly stochastic matrix M , there exists a set of $v \leq n^2 - 2n + 2$ ([60]) positive coefficients $\beta_i > 0$, with the property $\sum_{i=1}^v \beta_i = 1$, and a set of v permutation matrices P_i , such that their sum equals the original matrix M , that is, $\sum_{i=1}^v \beta_i \cdot P_i = M$.

B. Schedules, Appendix to Section 2

In this section, we more rigorously define the traffic schedule and give a few more helpful definitions regarding traffic schedules needed for our proofs. Lastly, we formally state the properties of a *feasible traffic schedule*.

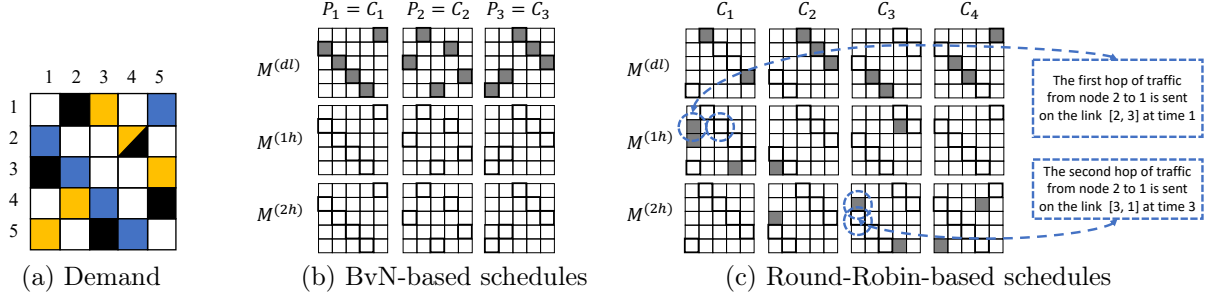


Figure 7: An example of demand and schedules, (a) Double stochastic demand matrix M . The value in a white cell is zero, cells with a single color have a value of $\frac{1}{3}$, and the dual-colored cell has a value of $\frac{2}{3}$. Each color (black, blue, or yellow) corresponds to a different permutation. (b) a simple BvN-decomposition-based schedules which decompose M into three permutation matrices (P_1, P_2, P_3) (that become the topology schedule (C_1, C_2, C_3)) and use only a single hop routing, i.e., $M^{(dl)}$. (c) a round-robin-based topology schedule (C_1, C_2, C_3, C_4) that is predetermined (and oblivious to the demand) and a traffic schedule that also uses 2-hops paths, $M^{(1h)}$, and $M^{(2h)}$.

Traffic Schedule. We begin by defining a *detailed traffic schedule*. A detailed traffic schedule defines the traffic sent at each configuration index i , i.e., during the switch configuration C_i . The detailed traffic schedule T_{pkg} is an ordered set of entries, which we assume are packets (though these could also represent traffic flows). Each entry τ_j is a tuple with the following six elements: (the configuration index at which the packet is sent, the packet size, the original source of the packet, the packet's final destination, the packet's current source, and the packet's current destination). We denote these elements as $\tau_j = \{t_j, w_j, s_j, d_j, x_j, y_j\}$. Since the packet is sent over configuration C_{t_j} , a valid entry must have $C_{t_j}[x_j, y_j] = 1$ or alternatively, the edge (x_j, y_j) is in the current matching C_{t_j} . In this work, we consider only paths of at most two hops. For a single hop path (s_j, d_j) , we denote the packet sent as *direct local* [14] and τ_j will be of the form $\{t_j, w_j, s_j, d_j, s_j, d_j\}$. For a two hops path $s_j \rightarrow q_j \rightarrow d_j$ we denote the packet sent in the first hop as *indirect local* [14] and τ_j will be of the form $\{t_j, w_j, s_j, d_j, s_j, q_j\}$. The packet sent in the second hop is denoted as *direct non-local* [14] and τ_j will be of the form $\{t_j, w_j, s_j, d_j, q_j, d_j\}$. To conclude, a detailed traffic schedule defines all traffic sent over a topology schedule.

The schedule represents the *amount* of traffic sent between each source and destination pair, and we use three sets of metrics to represent the traffic scheduling. This work considers only schedules that offer up to two hop paths, so we only need three different matrices to summarize the traffic at configuration index i : A matrix, $m_i^{(dl)}$ for the traffic sent via single hop (i.e., direct local), a matrix, $m_i^{(1h)}$ for the sent packets that are the first hop of a two hops paths traffic (i.e., local indirect), and a matrix, $m_i^{(2h)}$, for the packets that are second hop of a two hops traffic (i.e., direct non-local) traffic. Also note that in our model, we allow a

packet to traverse at most a single hop in each configuration C_i .

Next, we define these metrics formally. Let $\mathbb{1}_A$ be an indicator function for condition A . Every cell of $m_i^{(dl)}$, represents direct local traffic sent from s to d at configuration index i . $\forall s, d$ s.t. $C_i[s, d] = 1$,

$$m_i^{(dl)}[s, d] := \sum_{\tau_j} w_j \cdot \mathbb{1}_{t_j=i \wedge s_j=s \wedge d_j=y_j=d}, \quad (23)$$

and zero otherwise. Note that for this case if $m_i^{(dl)}[s, d] > 0$, then $C_i[s, d] = 1$. Next, every cell of $m_i^{(1h)}$ represents indirect local traffic, that is, traffic on its first hop out of two, at configuration index i . For each two hops path $s \rightarrow q \rightarrow d$, it update the cell $m_i^{(1h)}[s, d]$ when $C_i[s, q] = 1$. Formally, $\forall s, d$ s.t. $C_i[s, q] = 1$,

$$m_i^{(1h)}[s, d] := \sum_{\tau_j} w_j \cdot \mathbb{1}_{t_j=i \wedge s_j=s \wedge d_j=d \wedge y_j=q}, \quad (24)$$

and zero otherwise. Note that for this case if $C_i[s, d] = 1$, then $m_i^{(1h)}[s, d]$ is zero. Every cell of $m_i^{(2h)}$, represents direct non-local traffic. That is traffic on its last hop of two hops in configuration index i . For each two hops path $s \rightarrow q \rightarrow d$, it update the cell $m_i^{(2h)}[s, d]$ when $C_i[q, d] = 1$. Formally, $\forall s, d$ s.t. $C_i[q, d] = 1$,

$$m_i^{(2h)}[s, d] := \sum_{\tau_j} w_j \cdot \mathbb{1}_{t_j=i \wedge s_j=s \wedge x_j=q \wedge d_j=y_j=d}, \quad (25)$$

and zero otherwise. Note that for this case if $C_i[s, d] = 1$, then $m_i^{(2h)}[s, d]$ is zero. Formally, a traffic schedule, $\mathcal{S}_{\text{pkg}}$, is then defined as

$$\mathcal{S}_{\text{pkg}} = \{(m_1^{(dl)}, m_1^{(1h)}, m_1^{(2h)}), \dots, (m_v^{(dl)}, m_v^{(1h)}, m_v^{(2h)})\}.$$

Let $M^{(dl)}$, $M^{(1h)}$ and $M^{(2h)}$ be the ordered sets of the three types of matrices,

$M^{(dl)} = \{m_1^{(dl)}, \dots, m_v^{(dl)}\}$, $M^{(1h)} = \{m_1^{(1h)}, \dots, m_v^{(1h)}\}$ and $M^{(2h)} = \{m_1^{(2h)}, \dots, m_v^{(2h)}\}$. So, $M^{(dl)}$ is the set of the entire direct local schedule, $M^{(1h)}$ is the set of the entire indirect local schedule, and $M^{(2h)}$ is the set of the entire direct non-local schedule.

Note that the traffic scheduling explicitly defines what packets are sent in each configuration index i and on each link (k, l) s.t $C_i[k, l] = 1$, in particular, the *total* amount of bits that are sent on the link (k, l) at time i is: $\hat{m}_i[k, l] = m_i^{(dl)}[k, l] + \sum_d m_i^{(1h)}[k, d] + \sum_s m_i^{(2h)}[s, l]$. That is the sum of direct, first-hop, and second-hop traffic. We define $\hat{M}$ as the *total traffic matrix*:

Definition 8 (Total traffic matrix). *Given a traffic schedule, $\mathcal{S}_{\text{pkg}}$, we define the total traffic matrix $\hat{M}$ as a matrix where each cell $\hat{M}[k, l]$ denotes the total amount of bits sent on the link (k, l) throughout the entire schedule: $\hat{M}[k, l] = \sum_i \hat{m}_i[k, l]$.*

Next, we state our formal definition for a *complete schedule*,

Definition 9 (Complete schedules). *Let M be a demand matrix and $\mathcal{S}_{\text{top}}$ and $\mathcal{S}_{\text{pkg}}$ a pair of feasible schedules. We say that $\mathcal{S}_{\text{top}}$ and $\mathcal{S}_{\text{pkg}}$ are complete schedules for M if all traffic in M is sent and arrives at the destination.*

$$\sum_{i=1}^v m_i^{(dl)} + \sum_{i=1}^v m_i^{(1h)} = \sum_{i=1}^v m_i^{(dl)} + \sum_{i=1}^v m_i^{(2h)} = M \quad (26)$$

That is, all traffic in M is sent from the sources and arrives at the destinations

Figure 7 presents an example for the above concepts. In (a) we show a double stochastic demand matrix M , the value in each cell is 0, $\frac{1}{3}$, or $\frac{2}{3}$ depending on the number of colors in the cell. The sum of each row and column is one. In Figure 7 (b), we describe a simple *topology schedule* (C_1, C_2, C_3) , for BVN-SYS. Each C_i is highlighted by thick cells. For each C_i we have $\alpha_i = \frac{1}{3}$. The *traffic scheduling* uses only single-hop paths, so $M^{(1h)}$, and $M^{(2h)}$ have only zero matrices and $m_i^{(dl)} = \alpha_i C_i$ for each configuration index i . In this case, $\mathcal{S}_{\text{top}}$ and $\mathcal{S}_{\text{pkg}}$ are *complete schedules* since $\sum_i m_i^{(dl)} = M$. The traffic scheduling of BVN-SYS, $\mathcal{S}_{\text{pkg}}$, supports only single-hop paths, which means each element in the direct local matrix equals $m_i^{(dl)}[k, l] = \beta_i$ where $C_i[k, l] = 1$ and zero otherwise. In other words, the traffic schedule is the same as the topology schedule in terms of active matrix cells. Since we only use direct local traffic, $M^{(1h)}$ and $M^{(2h)}$ are sets of zero matrices.

In Figure 7 (c), we present a round-robin topology schedule. This schedule (i.e., (C_1, C_2, C_3, C_4) shown in thick cells), is predetermined, oblivious to the demand and rotates between a set of permutations that creates a complete graph [14]. Also here $\alpha_i = \frac{1}{3}$. The traffic schedule for this case uses 2-hops paths, as seen in $M^{(1h)}$, and $M^{(2h)}$. For example, at configuration index 1, node 2 sends a packet to node 3 (toward node 1) as shown in $m_1^{(1h)}[2, 1]$, and in configuration index 3, node 3 forwards this packet to node 1, as shown in $m_3^{(2h)}[2, 1]$. We can see this by looking closely at the figure. On the first configuration C_1 , at the first hop matrix $m_1^{(1h)}$, the cell $m_1^{(1h)}[2, 1]$ is gray; this represents demand in the demand matrix M . This demand is sent using the links $[2, 3]$ (which has a thickened border). The demand from $M[2, 1]$ is forwarded to node 3 in the second hop. Looking at the second hop at C_3 , we can see that the cell $m_3^{(2h)}[2, 1]$ is gray; this represents the original demand, which is now sent using link $[3, 1]$ towards node 1, completing the data transfer. For another example, look at the first configuration C_1 , at the first hop matrix $m_1^{(1h)}$, at the gray cell $m_1^{(1h)}[3, 1]$. The traffic from $[3, 1]$ is forwarded to node 4. In the second hop on C_2 , again the cell $m_2^{(1h)}[3, 1]$ is grey, its data is now sent using link $[4, 1]$ towards node 1 completing the data transfer. In this case, $\mathcal{S}_{\text{top}}$ and $\mathcal{S}_{\text{pkg}}$ are also *complete schedules* since $\sum_i m_i^{(dl)} + \sum_i m_i^{(1h)} = M$.

Next, we define matrices that present the total indirect and direct traffic sent to each link on each configuration index.

Definition 10 (Indirect and direct total matrices). *For a given traffic schedule $\mathcal{S}_{\text{pkg}}$, with a subset of permutation list $\{C_1, \dots, C_v\}$, we define the i th indirect local demand matrix $\hat{m}_i^{(1h)}$ as an $n \times n$ matrix with the following property.*

$$\hat{m}_i^{(1h)}[k, j] := \begin{cases} \sum_{l=1}^n m_i^{(1h)}[k, l] & \text{if } C_i[k, j] = 1 \\ 0 & \text{otherwise} \end{cases}$$

Similarly, for a direct non-local demand matrix $\hat{m}_i^{(2h)}$.

$$\hat{m}_i^{(2h)}[k, j] := \begin{cases} \sum_{l=1}^n m_i^{(2h)}[l, j] & \text{if } C_i[k, j] = 1 \\ 0 & \text{otherwise} \end{cases}$$

The direct local demand matrix is the same as the equivalent direct local traffic schedule matrix,

$$\hat{m}_i^{(dl)} := m_i^{(dl)}$$

We can redefine the Total traffic matrix of [Definition 8](#) using the above matrices.

Definition 11 (Total traffic matrix). *Given a demand matrix M , and $\mathcal{S}_{\text{pkg}}$ schedule, we define the total traffic matrix $\hat{M}$ as the total amount of bits that were sent on each link,*

$$\hat{M} = \sum_{i=1}^v \hat{m}_i^{(dl)} + \sum_{i=1}^v \hat{m}_i^{(1h)} + \sum_{i=1}^v \hat{m}_i^{(2h)}$$

That is, it's the sum of all induced demand from direct and indirect traffic.

B.1. Feasible Traffic Schedule Properties

A *feasible* traffic schedule needs to hold several properties. Recall that a detailed traffic schedule is a sequence of communication instructions: $\tau_j = \{t_j, w_j, s_j, d_j, x_j, y_j\}$.

Property #1 - Admissibility. For each τ_j , let C_{t_j} be the active switch configuration at time t_j then $C_{t_j}[x_j, y_j] = 1$.

property #2 - time sufficiency. Transmission time is sufficient on each configuration. $\forall 1 \leq k, l \leq n$:

$$\alpha_i \geq \frac{\hat{m}_i[k, l]}{r}$$

Property #3 - Causality. For every traffic that is sent over 2-hops, every first hop must be followed by a second hop at a later time, and every second hop must have a first hop at an earlier time.

It follows from property #3 that $m_1^{(2h)} = 0_n$ and $m_v^{(1h)} = 0_n$.

Another definition used in previous works is the ϵ schedule, a feasible but non “complete” schedule.

Definition 12 (ϵ schedules). *Given a demand matrix M , we denote a pair of $\mathcal{S}_{\text{top}}$ and $\mathcal{S}_{\text{pkg}}$ as ϵ schedules if the schedules are feasible, but send all traffic but an $\epsilon \geq 0$ Frobenius fraction of it.*

Formally,

$$\|M - \left(\sum_{i=1}^v m_i^{(dl)} + \sum_{i=1}^v m_i^{(1h)} \right)\|_F \leq \epsilon$$

Where $\|\cdot\|_F$ is the Frobenius norm.

C. Proofs for Sections 4

In this section, we prove the bounds we discussed in [Section 4](#). We first prove the bound for the direct traffic scheduler for RR-SYS.

C.1. Proofs for [Theorem 1](#)

We begin by defining a useful concept, the *traffic skewness* of $\mathcal{S}_{\text{pkg}}$, denoted as $\phi(\mathcal{S}_{\text{pkg}})$ (or ϕ for short), which we note was already defined similarly in Cerberus [16]. However, here, we will see how it can be naturally and formally defined using our traffic schedule framework. In short, $\phi(\mathcal{S}_{\text{pkg}})$ denotes the fraction of bits in RR-SYS, which are sent through a single hop, and $1 - \phi(\mathcal{S}_{\text{pkg}})$ is the fraction that is sent via two hops.

Definition 13 (The skewness of a traffic schedule). *Given a $\mathcal{S}_{\text{pkg}}$ the skewness of the schedule $\phi(\mathcal{S}_{\text{pkg}})$ is the ratio between directly sent traffic to all sent traffic, formally*

$$\phi(\mathcal{S}_{\text{pkg}}) = \frac{\|M^{(dl)}\|_{\Sigma}}{\|M^{(dl)}\|_{\Sigma} + \|M^{(1h)}\|_{\Sigma} + \|M^{(2h)}\|_{\Sigma}} \quad (27)$$

When the traffic schedule is *complete*, we can claim the following.

Lemma 11 (The skewness of a complete traffic schedule). *Let $\mathcal{S}_{\text{pkg}}$ be a complete traffic schedule (using [Definition 8](#)), then,*

$$\phi(\mathcal{S}_{\text{pkg}}) = 2 - \frac{\|\hat{M}\|_{\Sigma}}{\|M\|_{\Sigma}} \quad (28)$$

Where M and $\hat{M}$ are the demand matrix and the total traffic matrix, respectively.

Proof. Since $\phi(\mathcal{S}_{\text{pkg}})$ also can also be used to denote the average (per bit) path length for $\mathcal{S}_{\text{pkg}}$. As $\phi(\mathcal{S}_{\text{pkg}})$ is the fraction of bits sent at one hop and $1 - \phi(\mathcal{S}_{\text{pkg}})$ is the fraction sent at two, the average path length will be

$$1 \cdot \phi(\mathcal{S}_{\text{pkg}}) + 2 \cdot (1 - \phi(\mathcal{S}_{\text{pkg}})) = 2 - \phi(\mathcal{S}_{\text{pkg}})$$

In turn, since $\mathcal{S}_{\text{pkg}}$ is complete, every bit in M is sent via one or two hops, so the size of the total traffic matrix $\|\hat{M}\|_{\Sigma}$, is a function of the average path length that is

$$\|\hat{M}\|_{\Sigma} = (2 - \phi(\mathcal{S}_{\text{pkg}}))\|M\|_{\Sigma}$$

Moving terms gives us the lemma. □

Note that, by definition $0 \leq \phi(\mathcal{S}_{\text{pkg}}) \leq 1$. Since we use either one or two-hop traffic. In the rest of the paper, for simplicity, we will assume that the $\mathcal{S}_{\text{pkg}}$ is implicit and denote $\phi(\mathcal{S}_{\text{pkg}})$ as simply ϕ .

Another definition we state here is the *inactive cell*. Not every algorithm we consider will use all cells in the matrix to transmit data. We define an inactive cell in the following manner.

Definition 14 (Inactive cell). *Given a traffic schedule $\mathcal{S}_{\text{pkg}}$, and the total traffic matrix formed from it, a cell of $\hat{M}$ with the coordinates $[k, l]$ is inactive if it's never used to transmit traffic or formally (using [Definition 10](#)),*

$$\forall 1 \leq i \leq v, \hat{m}_i^{(2h)}[k, l] + \hat{m}_i^{(1h)}[k, l] + \hat{m}_i^{(dl)}[k, l] = 0 \quad (29)$$

That is, this cell is zero in all the total traffic schedule matrices.

One example of such cells is the cells of the diagonal, which in our model are always inactive.

We denote the set of all inactive cells in row i as $\omega_i^\emptyset$, and the total number of these in row i as $|\omega_i^\emptyset|$. Often, we will assume that the number of inactive cells in all rows is the same. We will denote this number as ω . That is $\forall i \in [1 \dots n], |\omega_i^\emptyset| = \omega$.

We note that since the total traffic matrix results from a traffic schedule, we can use it to determine a lower bound on the DCT. This is given by looking at the maximal element in $\hat{M}$.

Claim 1 (Total traffic matrix DCT). *Consider any demand matrix M and any algorithm $\mathcal{A}_{\text{pkg}}$. Let $\hat{M}$ be the total traffic matrix of the traffic schedule. The DCT of RR-SYS, is lower bounded by the maximal entry in $\hat{M}$*

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, M) \geq (n-1) \frac{\max(\hat{M}_{i,j})}{\eta r} \quad (30)$$

Proof. Since traffic in the total demand matrix is sent only directly (one hop) the DCT could be determined by using the DIRECT traffic scheduler on the total demand matrix $\hat{M}$, similarly as in [Theorem 6](#). $\square$

Note that this is a lower bound, as, at best, the maximal element of the total demand matrix $\max(\hat{M}_{i,j})$ is transmitted continuously, without pause, but this may not always be the case.

When $\mathcal{A}_{\text{pkg}}$ generates a schedule that transmits continuously on all connections, this is an exact result, that is, the DCT for such a schedule is the transmission time of the maximal entry.

To find the lower bound on the DCT of RR-SYS, we prove the following theorem.

Theorem 12. *[Lower Bound of DCT in RR-SYS] Let M be a demand matrix and $\mathcal{A}_{\text{pkg}}$ a traffic scheduling algorithm for RR-SYS that generates a complete $\mathcal{S}_{\text{pkg}}$ with ω inactive cells in each row. Then the RR-SYS demand completion time Δ is lower bounded as*

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, M) \geq (2-\phi) \frac{1}{\eta r} \frac{\|M\|_\Sigma}{n} \frac{n-1}{n-\omega} \quad (31)$$

Proof. We reach the results using the following derivation, which we explain below,

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, M) \geq \frac{n-1}{\eta r} \max(\hat{M}_{i,j}) \quad (32)$$

$$\geq \frac{n-1}{\eta r} \frac{\|\hat{M}\|_\Sigma}{n \cdot (n-\omega)} \quad (33)$$

$$= (2-\phi) \frac{1}{\eta r} \frac{\|M\|_\Sigma}{n} \frac{n-1}{n-\omega} \quad (34)$$

Eq. (32) follows from [Claim 1](#) that a lower bound on the DCT is the size of the maximal cell in the total traffic matrix $\hat{M}$. Eq. (33) is true since the largest cell is lower bounded by the average *active* cell size (active cells are all cells that are not inactive). That is, this is $\max(\hat{M}_{i,j}) \geq \frac{\|\hat{M}\|_\Sigma}{(n-\omega)n}$. Since $\mathcal{S}_{\text{pkg}}$ is complete, we can use our result for ϕ from [Lemma 11](#) in Eq. (34). That is, we use the following result, $\|\hat{M}\|_\Sigma = (2-\phi)\|M\|_\Sigma$, which gives us the last step of the theorem. $\square$

Note that the result from [Theorem 12](#) gives no bounds on the value of ϕ . Next, we prove an upper bound on ϕ for the $M(v)$ matrix family ([Section 2.4](#)), which we will use to prove the DCT.

Theorem 13 (Upper bound of ϕ on a $M(v)$). *Consider a demand matrix $M(v)$. Let $\mathcal{S}_{\text{pkg}}^*$ be any complete traffic scheduling for RR-SYS with ω inactive cells in each row, which minimizes the DCT for $M(v)$. Then $\phi^* = \phi(\mathcal{S}_{\text{pkg}}^*)$ is upper bounded by,*

$$\phi^* \leq \phi_{\min} = \frac{2v}{(n - \omega + v)} \quad (35)$$

Proof. Consider a complete schedule and its skewness ϕ . From Lemma 11 we have $\|\hat{M}\|_{\Sigma} = (2 - \phi)\|M\|_{\Sigma}$. From Claim 1 it follows that the lower bound for the DCT is minimized when the maximum cell in $\hat{M}$ is minimized. Next, we claim that since the schedule is optimal, each cell in $M(v)$ that has an entry $\frac{1}{v}$ will only send *direct* traffic. This follows from the observation that if it were needed to send an *indirect* traffic that travels two hops, it was better to send the same amount of traffic directly, reducing the overall traffic sent. Contradicting the optimality of the schedule.

Therefore, there are two types of cells in the total demand matrix $\hat{M}$: nv cells that send direct traffic and at most $n(n - \omega - v)$ cells that send *indirect* traffic. The total amount of direct traffic is $\phi\|M\|_{\Sigma} = \phi n$. The maximal cell that sends direct traffic is at least the average cell $\frac{n\phi}{nv}$. Similarly, the total amount of indirect traffic is $2(1 - \phi)\|M\|_{\Sigma} = 2(1 - \phi)n$. The maximal cell that sends indirect traffic is at least the average cell $\frac{2(1 - \phi)n}{n(n - \omega - v)}$. Therefore, the minimum of the maximum cell in $\hat{M}$ is achieved for $\phi_{\min}$,

$$\phi_{\min} = \arg \min_{\phi} \max\left(\frac{\phi}{v}, \frac{2(1 - \phi)}{n - \omega - v}\right).$$

The minimum ϕ is then when

$$\frac{\phi_{\min}}{v} = \frac{2(1 - \phi_{\min})}{n - \omega - v}$$

Which result in

$$\phi_{\min} = \frac{2v}{n - \omega + v}$$

□

We can now prove Theorem 1.

Theorem 1. [Lower bound for RR-SYS system DCT] *Let $\mathcal{A}_{\text{pkg}}$ be any traffic scheduling algorithm for RR-SYS that generates a complete schedule. Let $P \in P_{\pi}$ be any derangement permutation matrix, then we have*

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, P) \geq \left(2 - \frac{2}{n}\right) \frac{1}{\eta r} \frac{\|P\|_{\Sigma}}{n} \quad (5)$$

Proof. To prove the lower bound, we first set the number of inactive cells to $\omega = 1$. That is, only cells on the diagonal are inactive. now, we can state the lower bound for RR-SYS as a function ϕ as

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, M) \geq (2 - \phi) \frac{1}{\eta r} \frac{\|M\|_{\Sigma}}{n}$$

Clearly since $\frac{1}{\eta r} \frac{\|M\|_{\Sigma}}{n} > 0$ we need to find an upper bound on ϕ . This bound is also true for matrices from the double stochastic matrix family $M(v)$.

Since, as we recall, the only inactive cells are on the diagonal, $\omega = 1$, we can state the upper bound for ϕ on the $M(v)$ matrix family as

$$\phi \leq \frac{2v}{(n - 1 + v)}$$

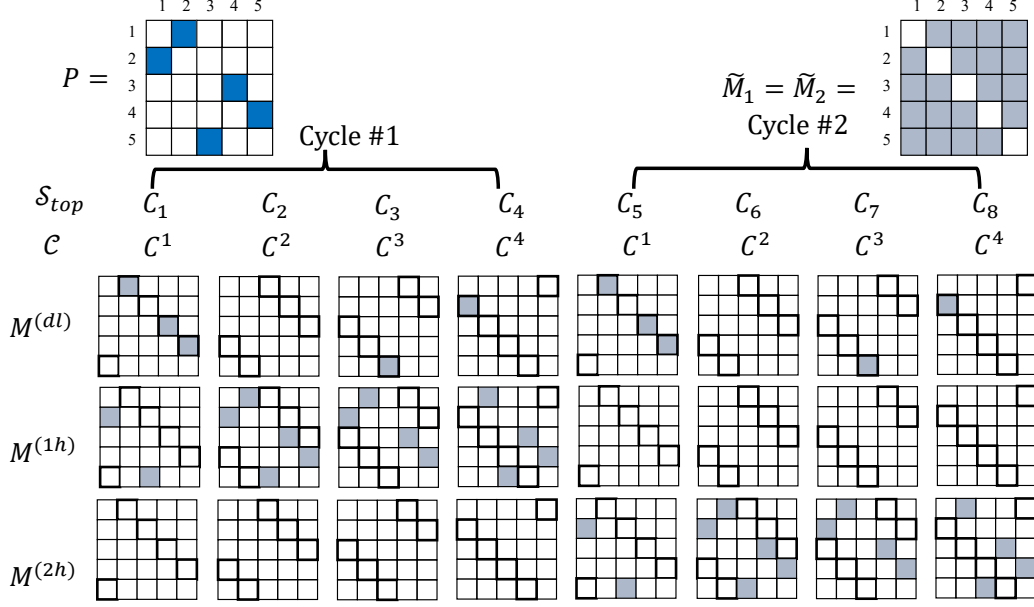


Figure 8: An example for a 5×5 demand permutation matrix P , and the traffic schedule generated for it, $\mathcal{S}_{\text{top}}$. The traffic schedule $\mathcal{S}_{\text{top}} = \{C_1, C_2, C_3, C_4\}$ is the same topology schedule as used in Figure 7. Each of the two cycles used in the schedule is marked. They generate the same total traffic matrix for each cycle $\tilde{M}_i$. Where each element marked in grey is equal to $\frac{1}{n} = \frac{1}{5}$, and the elements of the permutation matrix are marked by blue, are equal, and represent a traffic of 1

Setting $v = 1$ in the $M(v)$ matrix family results in the permutation matrix family. That $P_\pi = M(1)$, and thus, for any $P \in P_\pi$ we get

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, P) \geq \left(2 - \frac{2}{n}\right) \frac{1}{\eta r} \frac{\|M\|_\Sigma}{n}$$

This concludes the proof for the theorem. $\square$

C.2. The PERM Scheduler

In this section, we will discuss the Permeation Scheduler or, as we denote it, PERM. This algorithm generates a detailed traffic schedule for the permutation matrix, which, as we will later see, is optimal. In Algorithm 2, we see the pseudo-code for the scheduler. This scheduler receives a single permutation demand matrix P and a cycle configuration $\mathcal{C}$ for RR-SYS and uses them to generate a detailed traffic schedule for the demand. We note that a detailed traffic schedule could be summarized as in Section 2.2. This topology schedule is composed of two cycles, that is $v = 2n - 2$. The first $n - 1$ elements in $\mathcal{S}_{\text{top}}$ belong to the first cycle, and the last $n - 1$ elements belong to the second. Lastly, recall that the union of permutation matrices of the cycle configuration $C^i \in \mathcal{C}$ is a complete graph.

Before we begin describing the algorithm, we define a tool that will be useful in the pseudo-code of the algorithms. For a permutation matrix P we denote with the vector $\pi^{(P)} = \{\pi_1, \pi_2, \dots, \pi_n\}$ the indices of each non zero element in the matrix P , where for each row k in P , it holds that $P[k, \pi_k] = 1$.

The traffic schedule generated from PERM is composed of two equal “phases” each corresponding to a single cycle, in turn, corresponding to the two nested *For* loops in the pseudo-code, which start at Line 2 and Line 12 of the pseudo-code. Since P is a permutation matrix, for each row k in P we have one cell with demand (whose indexes are denoted by π_k) and $n - 2$ empty cells, which could be used for indirect routing (other than those on the diagonal). Inside the two nested *For* loops, we see how the algorithm redistributes $\frac{(n-2)}{n}$ bits of traffic from the demand cell in each row towards the empty cells. For example, in the first nested *For* loop in Line 5, we see that the algorithm generates an entry for direct traffic. This will happen exactly n times per run of the nested *For* loop, once per each element in P . In Line 7, we see an indirect local traffic entry for the empty cells of which there will $n(n - 2)$ instances. This is similar to the second cycle. However, here, instead of generating $n(n - 2)$ indirect local entries, we generate $n(n - 2)$ direct non-local traffic entries. We can see that for all entries in the schedule, their weight (w) is set to $\frac{1}{n}$. Since there are $n - 2$ empty cells and one demand cell on each row, each cell in the first and last cycle will send $\frac{1}{n}$ bits of data together each cell will send $\frac{2}{n}$ bits during the course of the entire schedule.

To help us understand the result of this algorithm, we can first summarize the detailed traffic schedule as in Section 2.2, and then look at it from a cycle perspective. To generate the first total demand *cycle* matrix, which we denote as $\tilde{M}_1$, we take the first $n - 1$ indirect and direct demand matrices and summarize them similarly to Definition 11.

$$\tilde{M}_1 = \sum_{i=1}^{n-1} \hat{m}_i^{(dl)} + \sum_{i=1}^{n-1} \hat{m}_i^{(1h)} + \sum_{i=1}^{n-1} \hat{m}_i^{(2h)}$$

We construct $\tilde{M}_2$ similarly, using the following set of $n - 1$ matrices in the schedule. Since the algorithm will send $\frac{1}{n}$ bits on each connection once per cycle, we know that each element in each matrix is equal to $\frac{1}{n}$. Therefore, each of the total demand cycle matrices is of the form $\tilde{M}_1 = \tilde{M}_2 = \frac{1}{n}(J_n - I_n)$. Where J_n is an all ones $n \times n$ matrix and I_n is the $n \times n$ identity matrix. In Figure 8, we see an example of a schedule generated from algorithm PERM on a 5×5 permutation matrix P . The traffic schedule is composed of a set of four permutation matrices, $\{C_1, C_2, C_3, C_4\}$, the same we use in Figure 7, they are denoted by the thicker border inside the schedule matrices. Looking at the two equal total traffic matrices for the two cycles $\tilde{M}_1, \tilde{M}_2$, we see that eventually, PERM distributes the permutation evenly among all connections. The schedule itself will use the first cycle to send all indirect local traffic, $M^{(1h)}$, and keep $m_i^{(dl)} = 0_n$, when $i \leq 4$, and the second to send direct nonlocal traffic, $M^{(2h)}$, and keep $m_i^{(1h)} = 0_n$, when $i \geq 4$. Both cycles are used to send direct local traffic $M^{(dl)}$. We further note that the colored squares of each matrix in the schedule $M^{(dl)}, M^{(1h)}, M^{(2h)}$, and in $\tilde{M}_1, \tilde{M}_2$ represent a (normalized) demand of $\frac{1}{5}$ bits. After describing the algorithm, we prove the following result for PERM.

Lemma 3. [*DCT for the PERM Algorithm*] Let $P \in P_\pi$ be an $n \times n$ scaled permutation matrix. Then, the schedule generated from the algorithm $\mathcal{A}_{\text{pkg}} = \text{PERM}$, is feasible, complete, and has a demand completion

time, Δ of,

$$\Delta(\text{RR}, \text{PERM}, P) = (2 - \frac{2}{n}) \frac{1}{\eta r} \frac{\|P\|_{\Sigma}}{n} \quad (7)$$

Proof. To prove this bound, we first prove that the schedule generated is complete, then we prove it is feasible, and lastly, we calculate the DCT of this schedule. For brevity and simplicity, we will assume that P is saturated and double stochastic and that the cycle hold time is $\eta = 1$, and that the transmission rate is normalized $r = 1$, that is, we set $\frac{\|P\|_{\Sigma}}{\eta n r} = 1$.

First, recall that there are two cycles in the $\mathcal{S}_{\text{pkg}}$ generated by the algorithm, which means we have $2n - 2$ elements in $\mathcal{S}_{\text{pkg}}$ in total. We show that $\mathcal{S}_{\text{pkg}}$ is complete since it satisfies [Definition 9](#) of a complete schedule. Summarizing the total direct and indirect local traffic matrices as in [Section 2.2](#) we get

$$\sum_{k=1}^{2n-2} m_k^{(dl)} + \sum_{k=1}^{2n-2} m_k^{(1h)} = P$$

To see why this is the case, let us look at the first nested *For* loop in the pseudo-code of [Algorithm 2](#) in Line 2. We see that the loop runs exactly once for each different connection of the first cycle, as this loop is used once for each C^i and once per connection in each C^i . In total, for each node k , we will have $(n - 1)$ entries (τ) generated. One entry for the direct traffic and $(n - 2)$ entries for indirect local traffic. In the second *For* loop in the pseudo-code of [Algorithm 2](#) in Line [Algorithm 12](#) we similarly have 1 entry per node of direct traffic and zero indirect local traffic in this cycle. Since the weight of each entry is $\frac{1}{n}$ we have a total weight of $\frac{1}{n} + \frac{1}{n} + \frac{n-2}{n} = 1$ per node, as each Line of P . Since every entry is generated with the following source and destination fields: $s = k$ and $d = \pi_k(P)$, which are the same as P for every node k , that is, for every line in P , we have our proof for completeness.

We must look at the properties described in [Section B.1](#) to show feasibility. To prove the first property, property #1, we need to prove that each entry τ is admissible. Since we built the schedule using only cells where $C^i[k, j] = 1$, for all $C^i \in \mathcal{C}$, that is, only for connected matching, as we also set $t = i$ for every traffic entry τ , as we see in lines [5](#), [7](#), [18](#) and [18](#), of the pseudo-code of [Algorithm 2](#), this propriety thus holds trivially.

For property #2, In lines [5](#), [7](#), [18](#) and [18](#), of [Algorithm 2](#) we see that each entry has exactly $\frac{1}{n}$ bits. Since the size of the transmission time for each switch configuration is also $\frac{1}{n}$ seconds (recall that $r = 1$) property #2 will hold with one assumption: That no two entries are sent on the same connection at same switch configuration C^i . To see if this is indeed the case, let us look at the first nested *For* loop in the pseudo-code of [Algorithm 2](#) in Line 2. We see that the loop runs exactly once for each connection of the first cycle, as this loop is used once for each C^i and once per connection in each C^i . In total, we will have $n(n - 1)$ entries τ generated. So, the number of entries is correct, but is each entry sent on a different connection during the cycle? Again, looking at the first nested *For* loop, each entry has a different source and destination (as k and j are always different), we see this is also correct. We can use the same arguments for the second nested *For* loop of the second cycle in Line [12](#). Thus, we can conclude that each connection will be used exactly once for each switch configuration to transmit $\frac{1}{n}$ bits, and therefore, property #2 holds.

The rest of the properties can be proven naively by looking at the construction of the schedule from two cycles. For propriety #3, this is true since, indeed, as we can see in the pseudo-code of [Algorithm 2](#), we see that all indirect local traffic is sent first. Therefore, this property holds, as there is exactly one packet sent in the first cycle, and in the second cycle, with the same weight.

Since the schedule is complete and feasible, we only need to calculate the duration of each cycle. Since, as we learn by looking at the total demand cycle matrices $\tilde{M}_1$, and $\tilde{M}_2$ each cell will send $\frac{1}{n}$ (normalized) bits in a cycle, we set δ to be $\delta = \frac{1}{n}$ seconds and each cycle has length of $(n - 1)\frac{1}{n}$ for a total of

$$2(n - 1)\frac{1}{n} = 2 - \frac{2}{n}$$

Algorithm 2 The PERM Algorithm

Require: $P \in \mathbf{R}^{+n \times n}$, $\mathcal{C}$ ▷ saturated permutation matrix and a RR cycle configuration,
 $\mathcal{C} = \{C^1, \dots, C^{n-1}\}$

Ensure: T_{pkg} ▷ A detailed traffic schedule (Section 2.2)

```
1:  $T_{\text{pkg}} = \{\}$ 
2: for all  $C^i \in \mathcal{C}$  do ▷ Generate the first cycle
3:   for all  $k, j \in C^i[k, j] = 1$  do
4:     if  $j = \pi_k(P)$  then
5:        $\tau = \{i, \frac{1}{n}, k, \pi_k(P), k, \pi_k(P)\}$  ▷ Direct traffic
6:     else
7:        $\tau = \{i, \frac{1}{n}, k, \pi_k(P), k, j\}$  ▷ Indirect traffic
8:        $T_{\text{pkg}} = T_{\text{pkg}}.\text{APPEND}(\tau)$ 
9:     end if
10:  end for
11: end for
12: for all  $C^i \in \mathcal{C}$  do ▷ Generate the second cycle
13:   for all  $k, j \in C^i[k, j] = 1$  do
14:     if  $j = \pi_k(P)$  then
15:        $\tau = \{i, \frac{1}{n}, k, \pi_k(P), k, \pi_k(P)\}$  ▷ Direct traffic
16:     else
17:        $\tau = \{2i, \frac{1}{n}, k, \pi_k(P), j, \pi_k(P)\}$ 
18:        $T_{\text{pkg}} = T_{\text{pkg}}.\text{APPEND}(\tau)$  ▷ Indirect traffic
19:     end if
20:   end for
21: end for
```

We can re-scale this expression by $\frac{\|P\|_{\Sigma}}{\eta n r}$. This means that each cell will send $\frac{1}{n} \cdot \frac{\|P\|_{\Sigma}}{n}$ bits, and we need to set δ to be $\delta = \frac{1}{n} \cdot \frac{1}{\eta r}$ seconds, to get our theorem. $\square$

C.3. The MULP Scheduler

In this section, we describe the MULP traffic scheduler algorithm. This can schedule any doubly stochastic matrix M and achieves a DCT, which we described in [Theorem 4](#). We give pseudo-code for this algorithm in [Algorithm 3](#). The algorithm requires a demand matrix and a RR topology schedule ($n - 1$ permutation matrices whose union creates a complete graph), The algorithm first creates a BvN decomposition from M , that is, it creates a permutation matrix list, $\{P_M = \{P_1, \dots, P_v\}$, and a coefficients list $\beta_M = \{\beta_1, \dots, \beta_v\}$. The algorithm then generates 'sub-schedules' $\mathcal{S}_{\text{pkg}}^*$ for each permutation matrix using the PERM algorithm. However, PERM requires a doubly stochastic matrix, which is why MULP will normalize each matrix P_i by β_i , that is, it will schedule $\frac{1}{\beta_i} P_i$. After receiving the schedule from PERM, the algorithm restores the demand to its original size. This can be done by, for example, multiplying each element in the traffic schedule by β_i , e.g. $\beta_i \cdot m_i^{(1h)}$. The algorithm then schedules each permutation sequentially, each time setting the slot hold time to $\delta_i = \frac{\beta_i}{r}$, where δ_i is the slot hold time for the i th sub-schedule. Since the sum total of all coefficients is one, $\sum_v \beta_i = 1$, the total DCT for any M on RR-SYS is the same as the DCT for a permutation matrix (as we've proved in [Theorem 4](#)).

Algorithm 3 The MULP Algorithm

Require: $M \in \mathbf{R}^{+^{n \times n}}$, $\mathcal{C}$ ▷ a doubly stochastic matrix and a RR cycle configuration
Ensure: $\mathcal{S}_{\text{pkg}}$ ▷ a traffic schedule
1: $\mathcal{S}_{\text{pkg}} = \{\}$
2: $P_M = \{P_1, \dots, P_v\}, \beta_M = \{\beta_1, \dots, \beta_v\}$ ▷ Decompose M to a set permutation matrices and coefficients using BvN
3: **for all** $i \in |P_M|$ **do**
4: $\mathcal{S}_{\text{pkg}}^* = \text{PERM}(\frac{1}{\beta_i} P_i)$ ▷ Create a schedule for $\frac{1}{\beta_i} P_i$ using PERM
5: re-scale the schedule the schedule $\mathcal{S}_{\text{pkg}}^*$ by β_i (i.e., By multiplying each element in the traffic schedule by β_i)
6: $\mathcal{S}_{\text{pkg}} \cup \mathcal{S}_{\text{pkg}}^*$
7: **end for**

C.4. Proofs for Theorem 6

Theorem 6. [DIRECT upper bound] Let M be any demand matrix and DIRECT be a traffic schedule where all packets are sent over a single hop. The matrix completion time in RR-SYS is then

$$\Delta(\text{RR}, \text{DIRECT}, M) = (n-1) \frac{\max(M_{i,j})}{\eta r} \quad (10)$$

Proof of Theorem 6. Recall that RR-SYS rotates between $n-1$ predefined matchings, which, when combined in a full cycle, form a complete graph. During a *slot*, there is a single active matching on which packets are sent. Since each source-destination pair appears only once in the complete graph, it has to wait for all other $n-2$ matchings to rotate until it is available again to send packets in a single hop. We start from the most general expression for DCT as stated in Equation 1, where $\mathcal{A}_{\text{pkg}} = \text{RR}$ and $\mathcal{A}_{\text{top}} = \text{DIRECT}$, we get.

$$\Delta(\text{RR}, \text{DIRECT}, M) = \sum_{i=1}^v (\alpha_i + R_i) = \sum_{i=1}^v (\delta + R_r) = v(\delta + R_r)$$

We want to find a closed-form formula for this result without v . Since, in this setting, we cannot send packets at more than one hop to transmit M , we need to wait for the network to send each entry on the demand matrix individually. Therefore, the DCT will be a function of the maximal entry of M , that is $\max(M_{i,j})$. Let us denote the number of complete cycles used to transmit M as x . In each cycle there are $n-1$ matchings use, and therefore $n-1$ periods of length δ used for transmission and $n-1$ reconfigurations of length R_r in each cycle, that is the DCT is, $\Delta(\text{RR}, \text{DIRECT}, M) = x(n-1)(\delta + R_r)$. Now, we need to find x . The number of cycles is the number of periods of length δ needed to transmit $\max(M_{i,j})$. This is simply the time to transmit the entire entry at full rate $\frac{\max(M_{i,j})}{r}$, to get x , we divide by δ , that is, $x = \frac{\max(M_{i,j})}{\delta r}$. We get

$$\Delta(\text{RR}, \text{DIRECT}, M) = v(n-1)(\delta + R_r) = \frac{(n-1) \max(M_{i,j})(\delta + R_r)}{\delta r}$$

And by setting $\eta = \frac{\delta}{\delta + R_r}$ the proof is complete. □

D. The Running Time of PIVOT

The running-time complexity of PIVOT is foremost a function of the BvN decomposition used, denoted as $O(\text{BvN})$ (and there exist many results in the literature as in [27]). Given a decomposition, we need to sort the v elements of the decomposition, $O(v \log(v))$. To calculate the DCT of BvN-SYS, we need to

look at the list of coefficients β_i with $O(v)$. To calculate the DCT of RR-SYS on its fraction of traffic, we need to find the maximum and mean elements of each sub-matrix. We, therefore, need to construct sub-matrices, adding or removing one permutation at a time, which, since a permutation is sparse, is an $O(n)$ operation. This operation can be combined with simple bookkeeping to keep track of the maximum and mean elements. The run time of the for loop is $vn + v$ in total. So the running-time complexity of the algorithm is $O(v(n + \log(v))) + O(\text{BVN})$.

E. Proofs and Analysis for Section 6

In Section 6, we analyzed the system DCT for COMP-SYS on the $M(v)$ matrix. In this section, we expand this analysis to the more general case $M(v, u)$, and for all ranges of R_b and n , we prove that the worst-case system DCT for all systems is at $u = 0$ we also state the expressions for the other two systems on $M(v, u)$.

Before starting this section, we discuss simple extensions to PIVOT and the UPPER traffic schedulers, denoted as PIVOT^+ and UPPER^+ , which will allow both to work optimally with the $M(v, u)$ matrix family when $u > 0$. For the systems RR-SYS and COMP-SYS, we will see that we can decompose the matrix $M(v, u)$ into two parts $uM(n - 1)$ and $(1 - u)M(v)$, and schedule each part independently with no negative effects on DCT. We first define PIVOT^+ as a pair of topology and traffic schedulers $\{\text{PIVOT}_T^+, \text{PIVOT}_P^+\}$. In this algorithm, we remove the uniform matrix component, $uM(n - 1)$, from the demand matrix (if such exists) and then schedule it to be served using RR-SYS. We then use PIVOT to schedule the remaining demand matrix, $(1 - u)M(v)$. Since $M(v, u) = (1 - u)M(v) + uM(n - 1)$ we serve all traffic. We define the DCT of such a system and algorithm as follows

Definition 15 (DCT of COMP-SYS with PIVOT^+).

$$\Delta(\text{PIVOT}_T^+, \text{PIVOT}_P^+, M) = \Delta(\text{RR}, \text{UPPER}, uM(n - 1)) + \Delta(\text{PIVOT}_T, \text{PIVOT}_P, (1 - u)M(v)) \quad (36)$$

Similarly, algorithm UPPER^+ , decomposes the matrix $M(v, u)$ into two parts $uM(n - 1)$ and $(1 - u)M(v)$ and schedules the uniform part optimally on RR-SYS with the DIRECT (as in Observation 2) algorithm, while it schedules $(1 - u)M(v)$ with UPPER as the traffic schedule. We note that when $u = 0$, both algorithms are the same as PIVOT and UPPER.

We also note that with BVN-SYS it would not be helpful to decompose the matrix in such a way. Since BVN-SYS cannot use RR-SYS type schedules, this will increase the number of reconfigurations and the DCT.

E.1. Worst Case Analysis for $M(v, u)$

In this section, we analyze the worst case for each system and show that it is at $u = 0$ as stated in Theorem 8.

To prove the worst case for BVN-SYS, we use BVN to decompose the matrix $M(v, u)$ into $n - 1$ permutations. $M(v, u)$ can be decomposed optimally into v permutations, each with a total load per row of

$\frac{1-u}{v} + \frac{u}{n-1}$, and another set of $n-1-v$ permutations each with a total load per row of $\frac{u}{n-1}$ combining these, will give us.

$$\Delta(\text{BVN}, \text{DIRECT}, M(v, u)) = \left(\frac{1-u}{v} + \frac{u}{n-1}\right)v + \frac{(n-1-v)(u)}{n-1} + (n-1)R_b = 1 + (n-1)R_b$$

Note that here, we assume a perfect BVN decomposition algorithm with full knowledge of the original v permutations. Since we have shown that there is no difference between the case of $M(v, u)$ for $u > 0$ and $1 \leq v \leq n-2$ and $M(n-1)$, this shows that for BVN-SYS, the worst case is found (also) at $u = 0$.

For RR-SYS we prove the following lemma.

Lemma 14 (worst case DCT for RR-SYS). *The worst case DCT for RR-SYS is for $u = 0$.*

$$\Delta(\text{RR}, \text{UPPER}^+, M(v, u)) \leq \Delta(\text{RR}, \text{UPPER}, M(v)) \quad (37)$$

Proof. With UPPER^+ for any $u > 0$ we decomposes $M(v, u)$ to $M(v, u) = (1-u)M(v) + uM(n-1)$. We then send each sub-matrix separately on RR-SYS. For any $v \geq 1$ and $0 \leq u \leq 1$ it holds that

$$\begin{aligned} \Delta(\text{RR}, \text{UPPER}^+, M(v, u)) &= \Delta(\text{RR}, \text{UPPER}, uM(n-1)) + \Delta(\text{RR}, \text{UPPER}, (1-u)M(v)) \\ &\leq \Delta(\text{RR}, \text{UPPER}, uM(v)) + \Delta(\text{RR}, \text{UPPER}, (1-u)M(v)) = \Delta(\text{RR}, \text{UPPER}, M(v)) \end{aligned}$$

The third line holds since:

$$\Delta(\text{RR}, \text{UPPER}, uM(n-1)) \leq \Delta(\text{RR}, \text{UPPER}, uM(v)),$$

which is true since $M(n-1)$ is sent optimally in with a DCT of 1 for any u and n .

The fourth line holds since UPPER will use the same algorithm (i.e., DIRECTOR MULP) in $M(v, u)$, $(1-u)M(v)$ and $uM(v)$ since the ratio between the maximum and the average cells is the same in all three matrices. In turn, the DCT equality follows. $\square$

Finally, for COMP-SYS we prove the following lemma.

Lemma 15 (worst case DCT for COMP-SYS). *The worst case DCT for COMP-SYS is for $u = 0$.*

$$\Delta(\text{PIVOT}_T^+, \text{PIVOT}_P^+, M(v, u)) \leq \Delta(\text{PIVOT}_T, \text{PIVOT}_P, M(v))$$

Proof. With PIVOT_P^+ , for any $u > 0$ we decomposes $M(v, u)$ to $M(v, u) = (1-u)M(v) + uM(n-1)$. We can send $uM(n-1)$ optimally on RR-SYS, incurring a DCT of u seconds. We get,

$$\Delta(\text{PIVOT}_T^+, \text{PIVOT}_P^+, M(v, u)) = u + \Delta(\text{PIVOT}_T, \text{PIVOT}_P, (1-u)M(v))$$

For $(1-u)M(v)$, recall that we have two options in accordance with [Observation 3](#): either we send $M(v)$ wholly with RR-SYS or with BVN-SYS. When using RR-SYS, we have a similar case as with [Lemma 14](#), since the matrix is sent entirely on RR-SYS, the lemma follows for this case. When using BVN-SYS, the DCT for the submatrix $(1-u)M(v)$ is $1-u + R_b v$. In total DCT for $M(v, u)$ on COMP-SYS in this case, therefore $u + 1 - u + R_b v = 1 + R_b v$, the same as the general case in [Lemma 14](#). As a result, the DCT will remain the same. Therefore, the lemma follows. $\square$

Following these lemmas, we note that the worst-case demand for all types of systems we study is $u = 0$.

E.2. System DCT of COMP-SYS on $M(v, u)$

In this section, we finalize the proof for [Theorem 8](#).

We begin by stating the generalized theorem for the DCT of COMP-SYS on $M(v, u)$ and the system DCT for the case of $R_b \geq \frac{(1-u)(2n-4)}{n^2}$. This theorem and its proof, therefore, will complete the result of [Theorem 8](#) for other values of R_b .

Theorem 16 (System DCT of COMP-SYS on $M(v, u)$ on the entire range of R_b). *Given M an $n \times n$ matrix $M \in M(v, u)$, the DCT of COMP-SYS is the following two-part expression partitioned by the value of the reconfiguration time R_b .*

First Case. $R_b < \frac{(1-u)(2n-4)}{n^2}$

$$\Delta(\text{PIVOT}_T^+, \text{PIVOT}_P^+, M) = \begin{cases} 1 + R_b v & 1 \leq v < (1-u) \frac{\sqrt{1+4R_b(n-1)}-1}{2R_b} \\ u + (1-u) \frac{(n-1)}{v} & \text{otherwise} \end{cases} \quad (38)$$

Second Case. $R_b \geq \frac{(1-u)(2n-4)}{n^2}$

$$\Delta(\text{PIVOT}_T^+, \text{PIVOT}_P^+, M) = \begin{cases} 1 + R_b v & 1 \leq v < \frac{(1-u)(n-2)}{nR_b} \\ u + (1-u)(2 - \frac{2}{n}) & \frac{n-2}{nR_b} \leq v < \frac{n}{2} \\ u + (1-u) \frac{(n-1)}{v} & \text{otherwise} \end{cases} \quad (39)$$

Now, let $\mathcal{M}$ be the family of all $M \in M(v, u)$ matrices. The system DCT of COMP-SYS is given when $u = 0$, and the first case was already stated in [Theorem 8](#), for completeness, we restate it here.

First Case. $R_b \leq \frac{(2n-4)}{n^2}$, the worst case is given for $v = \ddot{v}$ where $\ddot{v} = \frac{\sqrt{1+4R_b(n-1)}-1}{2R_b}$.

$$\Delta(\text{COMP-SYS}, \mathcal{M}) \leq \Delta(\text{PIVOT}_T^+, \text{PIVOT}_P^+, M(\ddot{v})) = 1 + \frac{\sqrt{1+4R_b(n-1)}-1}{2R_b} R_b \quad (40)$$

Second Case. $R_b > \frac{(2n-4)}{n^2}$, the worst case is given for $v = \dot{v}$ where this value is $\dot{v} = \frac{n-2}{nR_b}$. For this value of v the DCT is the same as the worst case DCT of RR-SYS from [Theorem 8](#).

$$\Delta(\text{COMP-SYS}, \mathcal{M}) \leq \Delta(\text{PIVOT}_T^+, \text{PIVOT}_P^+, M(\dot{v})) = \Delta(\text{RR}, \text{UPPER}, M(\dot{v})) = 2 - \frac{2}{n} \quad (41)$$

Proof. First, we show how to find the expressions for the DCT of COMP-SYS, we then prove what is the system DCT in each case. In COMP-SYS we decompose into two matrices. Using the PIVOT^+ , since RR-SYS can handle $M(n-1)$ optimally, we always send this sub-matrix using RR-SYS. For the other $M(v)$ matrix, use the PIVOT algorithm.

To define the DCT of COMP-SYS in a piecewise formula, we need to find the v value where the different components of COMP-SYS intersect and then use the appropriate expression for their DCT. The DCT for COMP-SYS on $M(v, u)$ is therefore

$$\Delta(\text{PIVOT}_T^+, \text{PIVOT}_P^+, M(v, u)) = u + \Delta(\text{PIVOT}_T^+, \text{PIVOT}_P^+, (1-u)M(v)) \quad (42)$$

Note that since the total traffic load here is $1-u$, the DCT for each subsystem is slightly different. For systems with a different load size, x (i.e., $x \neq 1$), the DCT of BVN-SYS on xM where M is a double stochastic demand matrix with a BVN decomposition of length v is, $\Delta(\text{BVN}, \text{DIRECT}, xM) = x + vR_b$. While for RR-SYS we get $\Delta(\text{RR}, \text{UPPER}, xM) = x\Delta(\text{RR}, \text{UPPER}, M)$, we, therefore, place these expressions in a piecewise expression as a function of v and R_b . We can use [Figure 2](#) as a visual aid in our explanation, as the curves

we are interested in remain similar in principle. To find the crossing point for RR-SYS with BVN-SYS, we have two cases: either the BVN-SYS crosses the RR-SYS, i.e., the UPPER curve at $u + (1 - u)(\frac{n-1}{v})$ (that is, between point (B) to (E)), or $u + (1 - u)(2 - \frac{2}{n})$ (that is, between point (D) to (B)). This crossing point area is a function of R_b , which changes for the BVN-SYS curve with (B) being the point of discontinuity. At this point the BVN reconfiguration time is $R_b = \frac{(1-u)(2n-4)}{n^2}$.

In the The first case for R_b values smaller than $R_b \leq \frac{(1-u)(2n-4)}{n^2}$. We denote the value of v for the cross point where $1 + R_b\ddot{v} = (1 - u)\frac{n-1}{\ddot{v}}$ as $\ddot{v}$. After solving for $\ddot{v}$ we get $\ddot{v} = (1 - u)\frac{\sqrt{1+4R_b(n-1)}-1}{2R_b}$.

The second case happens for $R_b > \frac{(1-u)(2n-4)}{n^2}$. The BVN-SYS DCT curve will cross the RR-SYS DCT curve at $u + (1 - u)(2 - \frac{2}{n})$. We denote the value of v for the second crossing point as $\dot{v}$, here we get $1 + R_b\dot{v} = (1 - u)(2 - \frac{2}{n})$. After solving for $\dot{v}$ we get $\dot{v} = (1 - u)\frac{n-2}{nR_b}$.

Opening the expression in Equation 42 using our formulas for the DCT of RR-SYS and BVN-SYS in a piecewise manner and using our different cases for R_b will give us the first part of the proof Theorem 16, namely, the expressions for the DCT. We now continue with proving the system DCT of COMP-SYS.

Since the worst case is at $u = 0$ (as shown in Section E.1), for COMP-SYS, the worst-case matrix is a function of n and R_b . We have two cases as a function of the BVN reconfiguration time R_b .

In the first case, $R_b \leq \frac{2n-4}{n^2}$, The cross point is at $\ddot{v} = \frac{\sqrt{1+4R_b(n-1)}-1}{2R_b}$, therefore

$$\Delta(\text{COMP-SYS}, \mathcal{M}) \leq \Delta(\text{PIVOT}_T, \text{PIVOT}_P, M(\ddot{v})) = 1 + \frac{\sqrt{1+4R_b(n-1)}-1}{2R_b} R_b \quad (43)$$

For the second case, where $R_b > \frac{2n-4}{n^2}$, COMP-SYS will cross the path of UPPER algorithm where UPPER is using MULP, that is, where the DCT is at $2 - \frac{2}{n}$. In this case, the upper bound is the same value as RR-SYS, but at a different v value, we denote this value as $\dot{v} = \frac{n-2}{nR_b}$.

$$\Delta(\text{COMP-SYS}, \mathcal{M}) \leq \Delta(\text{PIVOT}_T, \text{PIVOT}_P, M(\dot{v})) = 2 - \frac{2}{n} \quad (44)$$

This finalizes the proof. $\square$

Combining the result from Section E.1, that the worst case matrix for $M(v, u)$ for all systems is at $u = 0$ with our recently proven Theorem 16, and our results regarding BVN-SYS, from Observation 1 and RR-SYS from Theorem 5 for the system DCT of both systems, we prove our main theorem of interest, that is Theorem 8.

F. Improving the Lower DCT Bounds on $M(v)$ for RR-SYS

In this section, we discuss the lower bounds for RR-SYS for the particular case of the $M(v)$ matrix family we defined in Section 2.4. Recall that in this doubly stochastic matrix family, the demand matrix is composed of the summation of a set of v equal and non-overlapping permutation matrices. This family, as we will later see, presents both the worst and best-case matrices for RR-SYS, and thus, we believe it allows us to explore a wide range of possible performances for the RR-SYS type systems. While the bounds we present here are not strictly necessary for the analyses of the system DCT, we present this section as an independent contribution to the chapter.

Recall the lower bound for RR-SYS on $M(v)$ in Corollary 2. We note that the question of What is a tight lower bound for any matrix is still an open question. However, we believe that this lower bound is achievable

by some algorithm for all values of v , where $2 \leq v \leq n - 2$, but only for some configurations of permutations in $M(v)$. In the following section, we show that our lower bound is not tight even for $M(2)$. We show that there exist some matrices in $M(2)$ which do not allow DCT lower than the upper bound for any RR-SYS type traffic schedule. These matrices have a property we denote as a dual collision, as in [Definition 17](#).

F.1. Bounds For The $M(2)$ Matrix

In this section, we discuss bounds to the $M(2)$ family of matrices. Recall that according to [Corollary 2](#), the lower bound for $M(2)$ is $2 - \frac{4}{n+1}$. In the following theorem, we prove a better bound for some cases.

Theorem 17. *[Lower bound of $M(2)$ with a dual collision] For RR-SYS there exists a demand matrix $M \in M(2)$ that has, at least, two dual collision cells such that DCT for M is lower bounded by the upper bound in [Theorem 4](#) for any traffic scheduling $\mathcal{A}_{\text{pkg}}$. That is;*

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, M) \geq (2 - \frac{2}{n}). \quad (45)$$

We note that this is the same DCT given by MULP. This theorem shows that for some matrices from $M(2)$, the upper bound is optimal. However, we also present an example of a matrix from this family and a traffic schedule where MULP is not the optimal algorithm. We, therefore, conjecture that there is no scheduling algorithm $\mathcal{A}_{\text{pkg}}$ for RR-SYS that can reach the lower bound stated in [Corollary 2](#) for *any* matrix in $M(v)$. However, for some *particular* matrices in $M(v)$, the lower bound is tight by some feasible algorithm $\mathcal{A}_{\text{pkg}}$.

For the following proof of [Theorem 17](#), we first define a *collision cell*. This is a cell in the demand matrix that has a collision between direct non-local traffic (i.e., traffic on its second hop) and direct local traffic (i.e., traffic of a single hop). These collisions might limit what a routing algorithm could do in some instances. A collision occurs when there is a need to send two 'types' of traffic on the same link. We consider the case where these two types of traffic are direct non-local and direct local traffic. Furthermore, although this can be generalized, we define our collision for matrices from the matrix family $M \in M(v)$.

We are only interested in the relation between cells with demand and cells with zero demand in the demand matrix. That is, a collision is caused by traffic sent from a certain non-zero demand cell sending two-hop traffic, which, on its second hop, collides with traffic from another non-zero demand cell. Our definition of a *collision cell* is the following.

Definition 16 (Collision Cell). *Consider a demand matrix $M \in M(v)$ and a cell with positive demand $M[l, a] > 0$. An empty cell $M[l, j] = 0$ is a collision cell if $M[j, a] > 0$. More formally, the cell $M[l, j]$ is a collision cell, for the cell $M[l, a]$ if the following conditions holds*

$$M[l, a] > 0 \ \& \ M[l, j] = 0 \ \& \ M[j, a] > 0. \quad (46)$$

The collision will happen because if the link (l, j) were the first hop when 'helping' the cell $M[l, a]$, the link (j, a) must be the second hop in helping $M[l, a]$. But then, since the cell $M[j, a]$ also has positive demand, the link (j, a) will also need to help the demand $M[j, a]$ in a single-hop path.

The previous definition relates to a collision cell caused by a single other cell. An extension of this definition is a *dual collision cell*.

Definition 17 (Dual Collision Cell). *Given a demand matrix $M \in M(v)$, and two cells with demand $M[l, a] > 0$, $M[l, b] > 0$, an empty cell $M[l, j]$ is a dual collision cell if it has a collision both with $M[l, a]$ and $M[l, b]$. Or, more formally, $M[l, j]$ is a dual collision if the following conditions holds*

$$M[l, a] > 0 \ \& \ M[l, b] > 0 \ \& \ M[l, j] = 0 \ \& \ M[j, a] > 0 \ \& \ M[j, b] > 0. \quad (47)$$

Our observation here is that dual collision cells are often in pairs in $M(2)$.

Observation 6 (Pairs of Dual Collision Cell in $M(2)$). *If $M[l, j]$ is a dual collision cell in relation to $M[l, a]$, $M[l, b]$ in $M(2)$, then $M[j, l] = 0$, is also a dual cell in relation to $M[j, a] > 0 \ \& \ M[j, b]$.*

When $M[j, l]$ is also zero, we can see that it also must fulfill the condition $M[j, a] > 0 \ \& \ M[j, b] > 0 \ \& \ M[j, l] = 0 \ \& \ M[l, a] > 0 \ \& \ M[l, b] > 0$. The case $M[j, l] = 0$ will be true for any matrix $M \in M(2)$ since it has exactly two non-zero cells per line.

To find a matrix $M \in M(2)$ with two dual collision cells, we need to find a matrix with two rows with the same two columns with non-zero demand. We build this matrix M , from two permutation matrices, P_1 and P_2 , where $M = 0.5(P_1 + P_2)$. We can set $P_1[1, 2] = 1$, $P_1[n, n-1] = 1$ and $P_2[1, n-1] = 1$, $P_2[n, 2] = 1$. Other elements can be chosen arbitrarily, keeping with the $M(v)$ conditions as we described in [Section 6](#). In this case, the cells $M[1, n]$ and $M[n, 1]$ are dual collision cells.

After stating our definitions, we can prove [Theorem 17](#)

Proof of Theorem 17. The first part resembles our proof from [Theorem 13](#). Consider a complete schedule and its bound from [Claim 1](#). The lower bound for the DCT is minimized when the maximum cell in $\hat{M}$ is minimized. Hence, in this proof, we focus on rows with a dual collision cell, as even if some rows might allow for a lower DCT, rows with dual collision cells will dominate the demand completion time. Next, much like in [Theorem 13](#), we claim that since the schedule is optimal, the schedule will use a link (j, a) related to a cell $M[j, a]$ that has a positive entry (with demand equaling $\frac{1}{2}$) only to send a single hop traffic (i.e., direct local). This follows from the observation that if it was needed to use this link to send an *indirect* traffic (i.e., that travels two hops), it was better to send the same amount of traffic directly, thus reducing the overall traffic sent. Contradicting the optimality of the schedule.

Looking at the dual collision cells, there are only two fundamental ways in which any algorithm $\mathcal{A}_{\text{pkg}}$ can use the dual collision cells in the RR-SYS scheme: either it is not used, or it is used to send indirect traffic (i.e., the first of two hops).

For the first case, where $\mathcal{A}_{\text{pkg}}$ leaves these cells unused, the dual collision cells are inactive cells. To find the lower bound on DCT, in this case, we use the upper bound ϕ we found in [Theorem 13](#) and using the expression for DCT we found in [Theorem 12](#), for any algorithm $\mathcal{A}_{\text{pkg}}$ with $\omega = 2$ we find that the following bound.

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, M) \geq (2 - \frac{4}{n}) \frac{n-1}{n-2} \frac{1}{\eta r} \frac{\|M\|_{\Sigma}}{n} = (2 - \frac{2}{n}) \frac{1}{\eta r} \frac{\|M\|_{\Sigma}}{n}. \quad (48)$$

Which is the same bound as the general *upper* bound for $M(2)$ we found in [Theorem 4](#) by using the algorithm [MULP](#).

For the second case, $\mathcal{A}_{\text{pkg}}$ uses the dual collision cells (and their associated links) to send indirect traffic. We denote the two permutations that compose the matrix M as P_1 and P_2 . The algorithm uses the links related to dual collision cells to send x_1 bits from the cells of P_1 and x_2 bits of data from the cells of P_2 ,

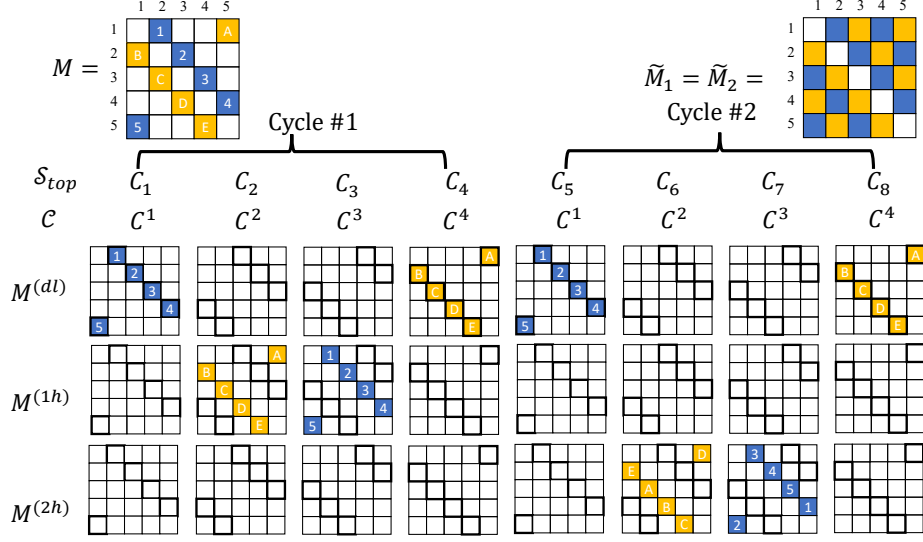


Figure 9: An example of a two-stage traffic schedule which reaches the lower bound from [Corollary 2](#), on a 5×5 demand matrix M , and the traffic schedule generated for it, $\mathcal{S}_{\text{top}}$. The traffic schedule $\mathcal{S}_{\text{top}} = \{C_1, C_2, C_3, C_4\}$ is the same topology schedule as used in [Figure 7](#). The permutation matrices composing M are marked with yellow and blue colors.

where $0 \leq x_1, x_2 \leq 0.5$. This would mean that (w.l.o.g) when the algorithm sends data from P_1 to P_2 via the dual collision cells, any data sent from P_1 to the dual collision cells will end up (according to [Definition 17](#), for a dual collision) being transmitted on connections that could have been used to transmit data from the cells of P_2 directly, in a single hop (and vice versa for P_2). That is, if $P_2[k, j] > 0$ then a bit sent to a dual collision cell would eventually have to be sent on the connection (k, j) , instead of direct local packets from P_2 . This means that by using the collision cells, we can send fewer bits directly from each permutation during the same period.

To prove that using the dual collision cells doesn't help to lower the DCT we prove that the following holds. First, we establish the amount of data sent in the base case, where the dual collision cells are inactive cells, that is, $x_1 = x_2 = 0$. Let $\hat{M}$ be a total traffic matrix formed from an optimal traffic algorithm that doesn't use the dual collision cell. Let $|P_1^T|$ and $|P_2^T|$ be the total demand sent on connections that are equivalent to P_1 and P_2 in the total traffic matrix.

If the algorithm, $\mathcal{A}_{\text{pkg}}$, transfers x_1 bits from the cells of permutation P_1 and x_2 bits from the cells of P_2 to the dual collision cells, then the total size of data transmitted on the connections, which are equivalent to P_1 is now $|P_2^T| - x_2 + x_1$ and for P_2 is now $|P_2^T| - x_1 + x_2$. Since the cells use the dual collision cells, any demand reduced from a cell in P_1 is transferred to a cell in P_2 (or vice versa). Lastly, we will assume w.l.o.g. that $x_1 \geq x_2$, and we can conclude that $|P_2^T| - x_1 + x_2 \geq |P_2^T|$, that is, the new transmission time is greater or equal to the original transmission time before using the dual collision cell for any value of x_1 and x_2 . Since we haven't been able to reduce the size of connections on the permutations, the size of the other connections is irrelevant. In conclusion, the total DCT for any algorithm $\mathcal{A}_{\text{pkg}}$ cannot be less than the bound we found for MULP. That is

$$\Delta(\text{RR}, \mathcal{A}_{\text{pkg}}, M) \geq \Delta(\text{RR}, \text{MULP}, M(2)) = (2 - \frac{2}{n}) \frac{1}{\eta r} \frac{\|M\|_{\Sigma}}{n}. \quad (49)$$

Since we have already proven that this bound is possible, the theorem follows. $\square$

F.2. Possible Optimal Traffic Schedule Example

The proof of [Theorem 17](#) shows that the upper bound we found is tight not just for all matrices from $M(1)$ and $M(n-1)$ but also for some matrices from $M(2)$, which contain a dual collision cell. However, we note that this criterion may not be the only one. We now show an example for a matrix $M \in M(2)$, which doesn't contain a dual collision but two (singular) collisions instead.

In [Figure 9](#), we show a traffic schedule ($\mathcal{S}_{\text{pkg}} = \{M^{(dl)}, M^{(2h)}, M^{(1h)}\}$) presenting a possible scheduling achieving the lower bound we found in [Corollary 2](#). Here, the topology scheduling used is the same as in [Figure 7](#). Looking at $M \in M(2)$, in the figure, we see that it is the sum of two permutation matrices, the one in blue marked with numbers (1 through 5), and the other in yellow marked with letters (A through E). These color markings are persistent throughout the figure. In the indirect local traffic set, $M^{(1h)}$, we see how the traffic from each matching is transferred to a different cell. While in the direct local set $M^{(2h)}$, we see the destination of each cell in the indirect first-hop traffic. Similarly to the schedule algorithm PERM, this schedule has two cycles, with two identical total traffic matrices in each cycle $\hat{M}_1$ and $\hat{M}_2$, each being uniform. In total, these would give us a traffic schedule that achieves the lower bound (from [Corollary 2](#)) where each matching cell has a normalized load of $\frac{1}{2}$ in M and a load of $\frac{1}{6}$ during the schedule. Finally, we set $\delta = \frac{1}{6}$ and see how this schedule is viable. The DCT of this schedule is $2 \cdot 4 \cdot \frac{1}{6} = \frac{4}{3} \approx 1.34[\text{sec}]$, that is, it the slot duration $\delta = \frac{1}{6}$ times the number of slots in a cycle, $n-1 = 4$ times the number of cycles which is 2. While if we used MULP, the result for this matrix M would have been $2 - \frac{2}{5} = 1.6[\text{sec}]$, and this is clearly higher. Note that the topology schedule we presented here in [Figure 9](#) contains the two permutations from M . However, this is not a requirement, and we could generate a schedule with the same DCT for another topology schedule.

Following the proof and lower bound example, we believe that all matrices from $M(v)$ have specific cases, that is, arrangements of the v matchings, where the matrix DCT is at the lower bound (from [Corollary 2](#)) and others where it is at the upper bound. and that this depends on the number of collisions. We leave proof of this for future work.

G. A Greedy Threshold-Based Algorithm

In this section, we discuss the GTN algorithm used in the empirical evaluation in [Section 7](#). We begin by describing it and defining its DCT. [Algorithm 4](#) presents **GreedyThresholdNet** (GTN), a natural and simple algorithm to minimize the DCT for COMP-SYS. Similarly to PIVOT, this algorithm decomposes the matrix into two sub-matrices, one for BVN-SYS and one for RR-SYS, and schedules each appropriately.

Given a saturated demand matrix M as input, the algorithm first decomposes M into a set of permutation matrices P_M and the corresponding coefficients set β_M using BVN. Then, for each P^* scaled permutation matrices where $P^* = \beta_i P_i$ we add P^* either to M^{BVN} if $\Delta(\text{BVN}, \text{DIRECT}, P^*) \leq \Delta(\text{RR}, \text{UPPER}, P^*)$ and otherwise we add it to M^{RR} . Since P^* is a scaled permutation matrix, we can easily calculate the *optimal*

Algorithm 4 The GreedyThresholdNet Algorithm

Require: $M \in \mathbf{R}^{n \times n}$ scaled double stochastic matrix

Ensure: $M^{(bvn)}, M^{(rr)}$

```
1:  $M^{(bvn)} = 0_n, M^{(rr)} = 0_n$  ▷ Initialize output
2:  $P_M = \{P_1, \dots, P_v\}, \beta_M = \{\beta_1, \dots, \beta_v\}$  ▷ Decompose  $M$  to a set permutation matrices and coefficients using BvN
3: for all  $P_i \in P_M$  do
4:    $P^* = \beta_i \cdot P_i$ 
5:   if  $\|P^*\|_\Sigma \geq \frac{nR_b}{n-2}$  then ▷ Compare traffic load of  $\|P^*\|_\Sigma$  to the threshold
6:      $M^{(bvn)} = M^{(bvn)} + P^*$ 
7:   else
8:      $M^{(rr)} = M^{(rr)} + P^*$ 
9:   end if
10: end for
```

DCT of each matching P^* on both RR-SYS and BvN-SYS and determine the partition of matrices to sub-systems. Instead of recomputing the DCT, we arrive at a simple *threshold*, $\frac{nR_b}{n-2}$, used in Line 5 and compare it to $\|P^*\|_\Sigma$, the load represented by the permutation P^* . More formally, this threshold can be computed in the following way,

$$\begin{aligned} \frac{\Delta(\text{BvN}, \text{DIRECT}, P^*)}{\Delta(\text{RR}, \text{UPPER}, P^*)} &\leq 1 \\ \frac{\|P^*\|_\Sigma + R_b}{(2 - \frac{2}{n})\|P^*\|_\Sigma} &\leq 1 \\ \frac{nR_b}{n-2} &\leq \|P^*\|_\Sigma. \end{aligned} \tag{50}$$

In other words, algorithm GTN decomposes traffic based on the threshold, $\frac{nR_b}{n-2}$, which is constant for any scaled permutation matrix given a the set of system parameters, n and R_b . As we saw in Section 7.2, even though this partition is locally (i.e., per permutation matrix) optimal, it does not give better results than PIVOT in the empirical setting. Similarly to PIVOT, in GTN, we schedule each sub-matrix generated from GTN on the appropriate system. This, again, generates an implicit pair of topology and traffic schedulers $\{\text{GTN}_T, \text{GTN}_P\}$. More formally, this can be described as the concatenation of schedules generated for each sub-matrix, M^{BvN} and M^{RR} . The DCT of COMP-SYS using GTN is therefore defined as follows.

Definition 18 (The DCT of COMP-SYS with GTN). *Given M a double stochastic scaled matrix, the DCT of COMP-SYS using GTN is*

$$\Delta(\text{GTN}_T, \text{GTN}_P, M) = \Delta(\text{BvN}, \text{DIRECT}, M^{\text{BvN}}) + \Delta(\text{RR}, \text{UPPER}, M^{\text{RR}}). \tag{51}$$

The **run-time analysis** of GTN is similar to PIVOT (as described in Appendix D). As in PIVOT, it includes $O(\text{BvN})$ time for the decomposition; however, GTN has a simpler loop. It does not need to sort matrices and test intermediate results. The Line 5 makes an $O(1)$ comparison for v times. The running time of GTN is therefore $O(v) + O(\text{BvN})$.

H. Comparison of PIVOT and GTN performance

In this section, we discuss the key differences between the two COMP-SYS decomposition algorithms evaluated in [Section 7](#), highlighting why and when PIVOT outperforms GTN.

GTN schedules traffic using a *static* threshold based on the performance of a *single* permutation matrix on either systems, it then schedules individual permutations with a load larger than the threshold on BvN-SYS and ones smaller to RR-SYS.

PIVOT differs from GTN, it takes a *global* and *dynamic* approach to scheduling traffic, which is not strictly *threshold-based*. PIVOT sorts permutations based on their load into an array, and selects the best ‘pivot permutation’, based on the performance (on either system) of the *accumulated* array of permutations to the left and the right of the pivot permutation. The selection of this pivot is *dynamic*, and the value of the load of the pivot permutation may change every time and is based on a *global* view of the traffic matrix. PIVOT cannot be reduced to a purely threshold-based algorithm other than in a specific case. When the BvN decomposition that PIVOT needs to sort (Line 1 of [Algorithm 1](#)) is one where each permutation corresponds to a different load (that is $\forall i, \beta_i \neq \beta_{i+1}$), it is possible to view the selected pivot permutation as implicitly defining a threshold. However, this interpretation is not accurate when some permutations represent identical loads (that is, if $\exists i, \beta_i = \beta_{i+1}$). In such cases, PIVOT can schedule permutations with identical loads separately on RR-SYS and BvN-SYS, something that is not possible in a threshold-based algorithm.

We give two examples that will illustrate these differing aspects. To demonstrate how PIVOT takes a global and dynamic view, we take the case where our traffic matrix M is a *perfectly uniform* matrix, that is, $M = M(n-1)$. M can be decomposed into a set of $n-1$ permutations, each representing an identical normalized load of $\frac{1}{n-1}$ bits. As described in [Algorithm 1](#), the PIVOT algorithm would sort the $n-1$ permutations by size, and then find the optimal partition of the permutations to two accumulated matrices M^{BvN} and M^{RR} . One such partition is where traffic is sent wholly to RR-SYS, that is, where $M^{\text{RR}} = M$ and $M^{\text{BvN}} = 0_n$. Since RR-SYS is optimal for uniform traffic matrices ([Observation 2](#)), PIVOT will schedule the entire traffic matrix M on RR-SYS. This would be the case regardless of parameters, as the DCT for RR-SYS is always optimal with a uniform matrix. In contrast, GTN will receive $n-1$ permutations, and, as described, in [Appendix G](#), it will use the threshold on every permutation separately. However, all permutations represent the same load, thus it will either schedule every permutation to RR-SYS or to BvN-SYS. If all traffic is scheduled with RR-SYS, then both PIVOT and GTN have the same DCT and throughput. However, when GTN schedules traffic with BvN-SYS COMP-SYS, using GTN would suffer a performance penalty. The specific action depends on the parameters of the system. Shortly, GTN schedules all traffic with BvN-SYS when $\frac{1}{n-1} \geq \frac{nR_b}{n-2}$, that is, when the condition on reconfiguration time of BvN-SYS holds, $R_b \leq \frac{n(n-1)}{n-2}$. The DCT for GTN in this case is $1 + (n-1)R_b$ while the DCT is 1 with PIVOT. This example demonstrates one

extreme end of GTN’s behavior; however, a similar behavior would occur whenever M is ‘close’ to a uniform matrix, even if not all permutations (resulting from a BVN decomposition on M) are identical.

Next, to illustrate a simple case where PIVOT schedules permutations with identical loads separately on RR-SYS and BVN-SYS, consider the following example. Suppose the BVN decomposition in PIVOT results in n permutations representing identical loads, and further assume that $n - 1$ form a uniform matrix (as $M(n - 1)$). In this scenario, PIVOT will sort the permutations arbitrarily. In one possible ordering, the $n - 1$ permutations forming the uniform matrix appear to the right (i.e., sent to RR-SYS) of the pivot permutation. This arrangement allows PIVOT to schedule the uniform matrix *optimally* on RR-SYS ([Observation 2](#)), while assigning the single remaining permutation to BVN-SYS. For brevity, we do not compute the exact conditions under which this occurs; however, it is more likely when R_b is small. This behavior cannot occur in any threshold-based algorithm, which would assign all permutations to one system or the other. Because PIVOT explicitly considers this possibility (Line 2 of the algorithm), and in this case chooses to schedule a single permutation on BVN-SYS, it will achieve superior performance. We note that PIVOT does not always enjoy this particular advantage. Nevertheless, this insight may inspire the design of improved algorithms in future works.